

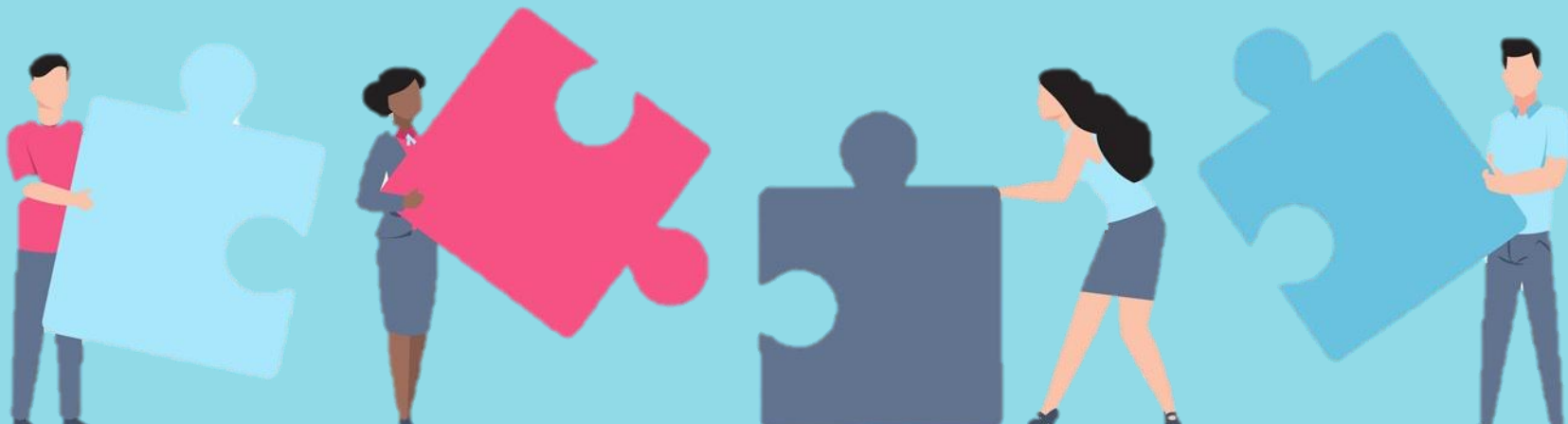
JavaScript-Get API



Ratiporn Chanklan

Array & Objects & JSON

- Objects คือตัวแปรที่สามารถเก็บค่าได้หลายค่า
- JSON ที่เป็นไฟล์ประเภทข้อความ เป็นรูปแบบข้อมูลที่เป็นมาตรฐานในการแลกเปลี่ยนข้อมูล



- อาร์เรย์ (Array) คือ

- ชุดของตัวแปรที่อยู่ในรูปลำดับใช้เก็บค่าข้อมูลให้อยู่ในกลุ่มเดียวกัน
- เป็นตัวแปรที่ใช้ในการเก็บข้อมูลที่มีลำดับที่ต่อเนื่อง
- ข้อมูลมีค่าได้หลายค่าโดยใช้ชื่ออ้างอิงได้เพียงชื่อเดียว
- ใช้หมายเลขกำกับ (index) ให้กับตัวแปรเพื่อจำแนกความแตกต่างของค่าตัวแปรแต่ละตัว

- การสร้าง Array

- สร้างโดยใช้เครื่องหมาย [] ข้อมูลแต่ละตัวคั่นด้วย , อ่านง่าย และนิยมใช้
 - let ชื่ออาร์เรย์ = []; กำหนดเป็นอาร์เรย์ว่าง (Empty Array)
 - let ชื่ออาร์เรย์ = [สมาชิกตัวที่1, สมาชิกตัวที่2, ...];
- สร้างโดยใช้คำสั่ง Array()
 - let ชื่ออาร์เรย์ = new Array(); กำหนดเป็นอาร์เรย์ว่าง (Empty Array)
 - let ชื่ออาร์เรย์ = new Array(n); จะหมายถึงความยาวของอาร์เรย์ n ตัว (ยังไม่มีค่า)
 - let ชื่ออาร์เรย์ = Array(สมาชิกตัวที่1, สมาชิกตัวที่2, ...);

- อาร์เรย์ผสมหลายชนิดข้อมูล

- JavaScript ไม่บังคับชนิดข้อมูลในอาร์เรย์สามารถเก็บได้ทั้งตัวเลข, string, boolean, object, array ซ้อน
- เช่น let mixed = [100, "Hello", true, {name: "John"}, [1,2,3]];

การเข้าถึงข้อมูลใน Array

- การเข้าถึงข้อมูลใน Array ของ JavaScript จะอ้างอิงตำแหน่ง (index) ของสมาชิก โดยเริ่มจาก 0
- ใช้ .length ในการหาจำนวนสมาชิกใน Array)

index	0	1	2
	apple	banana	mango

```
let fruits = ["apple", "banana", "mango"];

console.log(fruits[0]); // apple (ตัวแรก)
console.log(fruits[1]); // banana
console.log(fruits[2]); // mango
console.log(fruits[fruits.length - 1]); // mango
console.log(fruits.length); // 3
```

- index 0 = ตัวแรก → fruits[0]
- index 1 = ตัวที่สอง → fruits[1]
- index length - 1 = ตัวสุดท้าย → fruits[fruits.length - 1]

การแก้ไขค่าใน Array

5

- แก้ไขค่าใน Array สามารถแก้ไขโดยตรงด้วย index และใช้ = ในการกำหนดค่าให้ใหม่

```
let fruits = ["apple", "banana", "mango"];  
fruits[1] = "orange"; // แก้ "banana" → "orange"  
console.log(fruits); // ["apple", "orange", "mango"]
```

การ เพิ่ม/ลบ ข้อมูลใน Array

- เพิ่มค่าที่ตำแหน่งใหม่ ถ้ากำหนด index ที่ไม่มีใน array หมายถึงการสร้างสมาชิกใหม่ที่ตำแหน่งนั้น
 - แก้โดยตรง → ใช้ `array[index] = newValue`

```
let fruits = ["apple", "banana", "mango"];

fruits[3] = "grape";
console.log(fruits); // ["apple", "orange", "mango", "grape"]
```

การ เพิ่ม/ลบ ข้อมูลใน Array โดยใช้เมธอด Array

7

- push() → เพิ่มท้าย
- unshift() → เพิ่มหน้า
- pop() → ลบตัวท้าย
- shift() → ลบตัวหน้า
- splice() → เพิ่ม ลบ แก้ไข

```
let fruits = ["apple", "banana", "mango"];

fruits.push("grape"); // เพิ่มท้าย ["apple", "banana", "mango", "grape"];
fruits.pop();         // ลบตัวท้าย ["apple", "banana", "mango"];
fruits.unshift("kiwi"); // เพิ่มหน้า ["kiwi", "apple", "banana", "mango"];
fruits.shift();       // ลบตัวหน้า ["apple", "banana", "mango"];
```

การ เพิ่ม/ลบ ข้อมูลใน Array โดยใช้เมธอด Array

8

- splice() → เพิ่ม ลบ แก้ไข
- รูปแบบการใช้งาน `array.splice(start, deleteCount, item1, item2, ...)`
 - start คือ index ที่จะเริ่มทำงาน
 - deleteCount คือ จำนวนสมาชิกที่จะลบ
 - ลบ → ใส่ deleteCount > 0
 - เพิ่ม → ใส่ deleteCount = 0 และเพิ่ม item เข้าไป
 - แทนที่ → ใส่ deleteCount > 0 พร้อม item ใหม่
 - item1, item2, ... คือ ค่าที่จะใส่เพิ่มเข้าไป (optional)

```
let fruits = ["apple", "banana", "mango", "orange", "pineapple"];

// ลบ 2 ตัว เริ่มที่ index 1
fruits.splice(1, 2);
console.log(fruits); //["apple", "orange", "pineapple"]

// เพิ่มสมาชิกใหม่ ที่ index 2, ลบ 0 ตัว, เพิ่ม 3 และ 4
fruits.splice(2, 0, 3, 4);
console.log(fruits); //["apple", "orange", 3, 4, "pineapple"]

// แก้ไข (แทนที่สมาชิก) ที่ index 1, ลบ 1 ตัว, แทนที่ด้วย "yellow"
fruits.splice(1, 1, "yellow");
console.log(fruits); //["apple", "yellow", 3, 4, "pineapple"]

// ลบทั้งหมดตั้งแต่ index 2 เป็นต้นไปจนถึงท้าย
fruits.splice(2);
console.log(fruits); //["apple", "yellow"]
```

การเข้าถึงค่าใน array ด้วย ลูป

9

- ใช้ for

```
for (เริ่มต้น; เงื่อนไข; การเปลี่ยนค่า) {  
  // คำสั่งที่จะทำซ้ำ  
}
```

- เริ่มต้น → กำหนดค่าตัวแปรเริ่มต้น (เช่น let i = 0)
- เงื่อนไข → ถ้าเป็นจริง (true) จะวนซ้ำ ถ้าเป็นเท็จ (false) จะหยุด
- การเปลี่ยนค่า → ปรับค่าตัวแปรหลังจบแต่ละรอบ

```
let fruits = ["apple", "banana", "mango"];  
  
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

- ใช้ for...of เป็นวิธีการที่ไม่ต้องใช้ index จะวนซ้ำทีละ ค่า (value) ของสมาชิกในอาร์เรย์

```
for (let ตัวแปร of อาร์เรย์) {  
  // ทำงานกับค่าของแต่ละสมาชิก  
}
```

- ตัวแปร → ตัวเก็บค่า (value) ของสมาชิกในแต่ละรอบ
- อาร์เรย์ → ข้อมูลที่ต้องการวนซ้ำ

```
let fruits = ["apple", "banana", "mango"];  
  
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

การเข้าถึงค่าใน array ด้วย ลูป

10

- ใช้ forEach

```
array.forEach(function(ค่า, index) {  
    // ทำงานกับค่าในแต่ละรอบ  
});
```

- ค่า (value) → ตัวแปรที่เก็บ "ค่า" ของสมาชิกในอาร์เรย์
- index → ตำแหน่งของสมาชิก (เริ่มที่ 0)

```
let fruits = ["apple", "banana", "mango"];  
fruits.forEach(function(fruit, index) {  
    console.log(index, fruit);  
});
```

- เขียนแบบสั้นด้วย Arrow Function

```
let fruits = ["apple", "banana", "mango"];  
  
fruits.forEach((fruit, index) => {  
    console.log(index, fruit);  
});
```

การค้นหาค่าใน array

- indexOf() เป็นเมธอดของ Array ใช้ค้นหาตำแหน่ง (index) ของสมาชิกในอาร์เรย์
 - ถ้าเจอ → คืนค่า index (เริ่มนับจาก 0)
 - ถ้าไม่เจอ → คืนค่า -1
- includes() ใช้ตรวจสอบว่าในอาร์เรย์ (หรือสตริง) มีค่านั้นหรือไม่
 - คืนค่าเป็น true หรือ false

```
let fruits = ["apple", "banana", "mango"];  
console.log(fruits.indexOf("mango")); // 2  
console.log(fruits.includes("banana")); // true
```

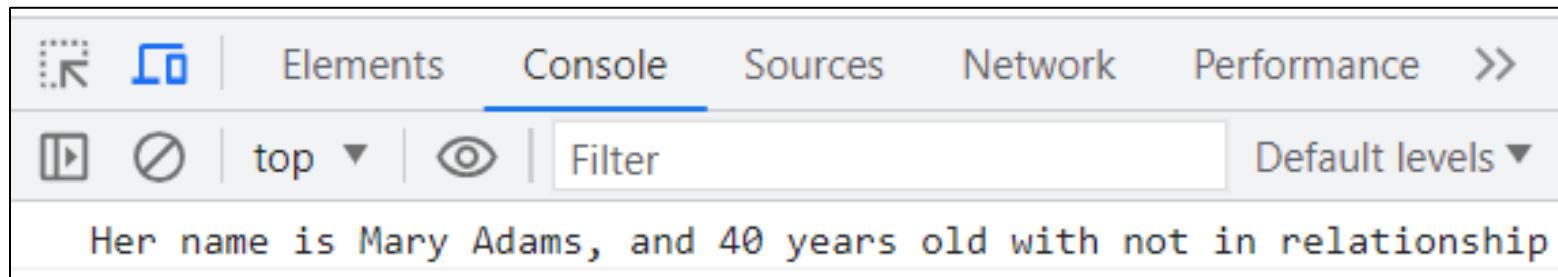
การประกาศตัวแปร

12

- โดยปกติ คน 1 คน ประกอบด้วย ชื่อ (firstName) นามสกุล (lastName) อายุ (Age) เกรด (Grade) สถานะ (inRelationship)
- หากต้องการเก็บข้อมูลของหลายคนก็ต้องประกาศตัวแปรเพิ่ม

```
const firstName = "Mary"
const lastName = "Adams"
const age = 40
const inRelationship = false

console.log(`Her name is ${firstName} ${lastName}, and ${age} years old with
${inRelationship? 'in relationship': 'not in relationship'}`)
```



- *เครื่องหมาย ` (backtick) เลือกภาษาที่พิมพ์ปัจจุบันเป็นภาษาอังกฤษ กด Windows + ปุ่ม ~ ค้างไว้ประมาณ 1 วินาที
- *Backticks ใน JavaScript นั้นหลัก ๆ ช่วยในเรื่องของการจัดการค่าในตัวแปรร่วมกับข้อความ (variable with string)

- ออบเจ็ค (Object) คือตัวแปรที่สามารถเก็บค่าได้หลายค่า
 - Object เป็นชนิดข้อมูล (data type) ที่เก็บข้อมูลแบบ คู่ key : value
 - แต่ละ key (หรือ property) จะชี้ไปยัง value ซึ่งอาจเป็นได้ทั้ง:
 - string, number, Boolean
 - array
 - object อื่น ๆ
 - function (method)



- ใน JavaScript สามารถสร้าง Object ได้ 3 วิธี ได้แก่

1. ใช้ Object Literal

- วิธีนี้เป็นวิธีที่ง่ายที่สุดในการสร้าง Object ทำได้โดยการเขียนลิสต์ของคู่ name:value ไว้ในวงเล็บปีกกา
- โดยเขียนอยู่ในรูปแบบของ Key: Value (ที่เรียกว่า Property (คุณสมบัติของออบเจ็กต์) โดยใช้เครื่องหมาย colon ในการแยกชื่อและค่าของมัน) และถ้าออบเจ็กต์มีหลาย Property สามารถคั่นแต่ละค่าด้วยเครื่องหมายคอมมา (,)
- โดยที่ Key นั้นจะเป็น String ส่วน Value สามารถเป็นข้อมูลประเภทใดๆ

```
const person = {  
  firstName: "Mary",  
  lastName: "Adams",  
  age: 40,  
  inRelationship: false  
}
```



- ออบเจ็ค (Object) คือตัวแปรที่สามารถเก็บค่าได้หลายค่า
- ใน JavaScript สามารถสร้าง Object ได้ 3 วิธี ได้แก่

2. ใช้คีย์เวิร์ด new

- สร้างออบเจ็คจากคลาส Object และกำหนด Property ให้กับออบเจ็คในภายหลัง
- ผลลัพธ์ที่ได้จากวิธีนี้เหมือนกับการใช้ Object Literal ทุกประการ ดังนั้นถ้าจะประหยัดเวลาเขียนโค้ดก็ใช้วิธีแรกดีกว่า

```
const person = new Object()  
  person.firstName = "Mary"  
  person.lastName= "Adams"  
  person.age = 40,  
  person.inRelationship = false
```

- ออบเจ็ค (Object) คือตัวแปรที่สามารถเก็บค่าได้หลายค่า
- ใน JavaScript สามารถสร้าง Object ได้ 3 วิธี ได้แก่

3. ใช้ object constructor function

- การสร้าง Object จากสองวิธีก่อนหน้านี้จะสร้างได้แค่ Object เดียวเท่านั้น
- ถ้าต้องการสร้าง Object ชนิดเดียวกันหลายๆตัว (มี Property เหมือนกัน) สามารถทำได้โดยใช้สิ่งที่เรียกว่า object constructor function
- เป็นสร้างคอนสตรัคเตอร์จากฟังก์ชัน หรือกล่าวได้ว่าข้อมูลทั้งหมดในฟังก์ชันเป็นเนื้อหาของคอนสตรัคเตอร์ที่ใช้สร้างเป็นวัตถุ
- คอนสตรัคเตอร์ คือ ฟังก์ชันชนิดหนึ่งที่มีไว้สำหรับกำหนดค่าเริ่มต้นให้กับตัวแปรคลาสของออบเจ็ค

- ออบเจ็ค (Object) คือตัวแปรที่สามารถเก็บค่าได้หลายค่า
- ใน JavaScript สามารถสร้าง Object ได้ 3 วิธี ได้แก่

3. ใช้ object constructor function

***this keyword เป็นตัวชี้ (reference) ไปยัง object ปัจจุบัน ใช้ภายใน method เพื่อเข้าถึง property อื่น ๆ ใน object เดียวกัน

```
function person(first,last,age,inRelationship ) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.inRelationship = inRelationship;  
}  
var mary = new person("Mary", "Adams",40, false);  
var bobby = new person("Bobby","Kidd", 19, true);  
  
console.log(`Her name is ${mary.firstName} ${mary.lastName}, and ${mary.age} years old with  
${mary.inRelationship? 'in relationship': 'not in relationship'}`)  
  
console.log(`Her name is ${bobby['firstName']} ${bobby['lastName']}, and ${bobby['age']}  
years old with ${bobby['inRelationship']? 'in relationship': 'not in relationship'}`)
```

เข้าถึง Property ของ Objects

18

- สามารถเข้าถึง Property ของออบเจ็กต์เพื่ออ่านค่าและเปลี่ยนแปลงค่า มี 2 วิธี คือ
 1. ด้วยวิธีของ Object notation ในรูปแบบ object.key

```
const person = {  
  firstName: "Mary",  
  lastName: "Adams",  
  age: 40,  
  inRelationship: false  
}
```

```
console.log(`Her name is ${person.firstName} ${person.lastName}, and ${person.age} years old with  
${person.inRelationship? 'in relationship': 'not in relationship'}`)
```



เข้าถึง Property ของ Objects

19

- สามารถเข้าถึง Property ของออบเจ็คเพื่ออ่านค่าและเปลี่ยนแปลงค่า มี 2 วิธี คือ
 2. ด้วยวิธีของ Bracket notation ในรูปแบบ object['key']

```
const person = {  
  firstName: "Mary",  
  lastName: "Adams",  
  age: 40,  
  inRelationship: false  
}
```

```
console.log(`Her name is ${person['firstName']} ${person['lastName']}, and ${person['age']} years old  
with ${person['inRelationship']}? 'in relationship': 'not in relationship'`)
```



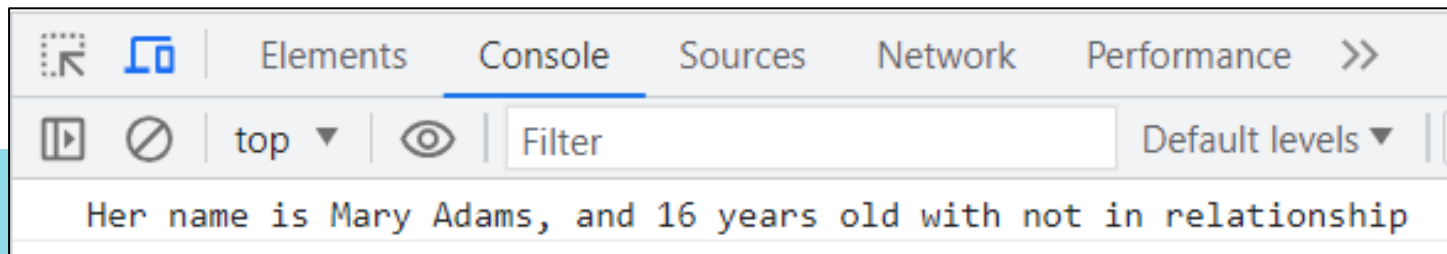
ภายใน Objects สามารถมี function ได้

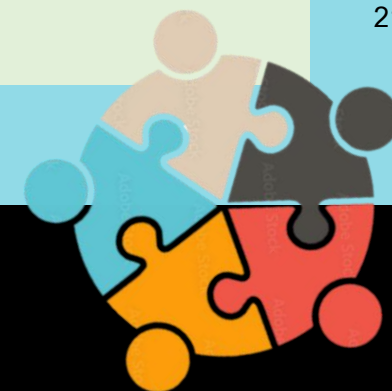
20

- Function ที่อยู่ภายใน Object เรียกว่า method

```
const person = {  
  firstName: 'Mary',  
  lastName: 'Adams',  
  YearOfBirth: 2000,  
  age: function() {  
    return (2016 - this.YearOfBirth);  
  },  
  inRelationship: false  
}
```

```
console.log(`Her name is ${person.firstName} ${person.lastName}, and ${person.age()}  
years old with ${person.inRelationship? 'in relationship': 'not in relationship'}`)
```





- เป็น Object ที่สามารถเรียกใช้งานได้โดยไม่ต้องสร้างเอง เช่น
- Date: เป็น Object เกี่ยวกับวันและเวลาสามารถสร้าง Object ด้วยคำสั่ง new เช่น

```
var x = new Date();
console.log(x)//Wed Aug 09 2023 15:32:39 GMT+0700 (Indochina Time)
console.log(x.toString().split(' ')[0]) //Wed
console.log(x.toString().split(' ')[1]) //Aug
console.log(x.toString().split(' ')[2]) //09
console.log(x.toString().split(' ')[3]) //2023
console.log(x.toString().split(' ')[4]) //15:32:39
```

```
var days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];
var dayName = days[x.getDay()];
console.log(dayName) //Wednesday
```

```
var options = { weekday: 'long', year: 'numeric', month: 'long', day: 'numeric' };
console.log(x.toLocaleDateString('th-TH', options)); //วันพุธที่ 9 สิงหาคม 2566
console.log(x.toLocaleDateString('en-EN', options)); //Wednesday, August 9, 2023
```

```
var options = { weekday: 'short', month: 'short', day: 'numeric', hour: 'numeric', minute: 'numeric', hour12: true};
console.log(x.toLocaleString('en-US', options)); //Wed, Aug 9, 4:04 PM
```

```
var day = x.getUTCDate();// ส่งค่ากลับเป็นวันที่ 1-31
var month = x.getUTCMonth(); // ส่งค่ากลับเป็นเดือน 0-11 ดังนั้น 0 คือ เดือนมกราคม
var year = x.getUTCFullYear(); // ส่งค่ากลับเป็นปี เช่น 2023
```

- เป็น Object ที่สามารถเรียกใช้งานได้โดยไม่ต้องสร้างเอง เช่น
- Math: เป็น Object เกี่ยวกับคณิตศาสตร์
 - `Math.round(4.7) // 5` เป็นการปัดเศษ ในกรณี ปัดขึ้น
 - `Math.round(4.1) // 4` เป็นการปัดเศษ ในกรณี ปัดลง
 - `Math.floor(4.7) // 4` เป็นการปัดเศษลง
 - `Math.ceil(4.1) // 5` เป็นการปัดเศษขึ้น
 - `Math.random() //` สุ่มตัวเลขระหว่าง 0 – 1 เช่น 0.10555, 0.8503
 - `Math.floor('2.24') // 2` เปลี่ยนข้อมูลเป็นตัวเลข



- JSON ย่อมาจาก (JavaScript Object Notation)
- JSON ที่เป็นไฟล์ประเภทข้อความ (Text based) ขนาดเล็ก
- เป็นรูปแบบข้อมูลที่เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลระหว่างเซิร์ฟเวอร์และไคลเอนต์ที่ได้รับความนิยมในปัจจุบัน
- อักขระมาตรฐานของ JSON
 - เครื่องหมาย : ใช้สำหรับแยกค่า key และ value
 - เครื่องหมาย , ใช้สำหรับแยกข้อมูล key-value ในแต่ละคู่
 - เครื่องหมาย { และ } ระบุว่าข้อมูลเป็นออบเจ็กต์
 - เครื่องหมาย [และ] ระบุว่าข้อมูลเป็นอาร์เรย์
 - เครื่องหมาย “” (double quotes) ใช้เขียนค่า key-value ใน JSON

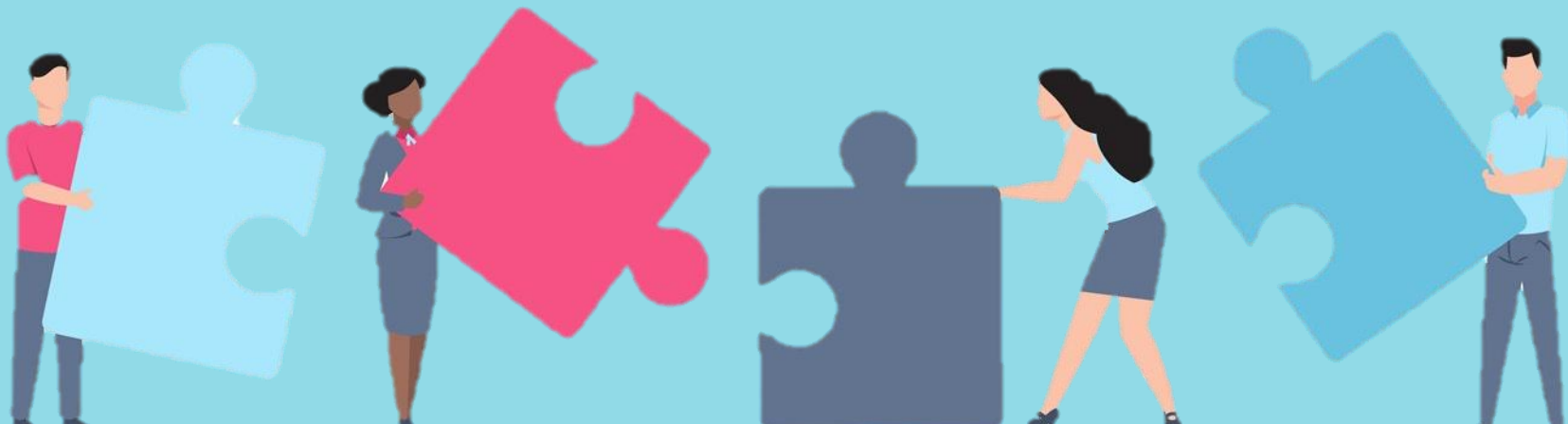
```
const json = {  
  "String": "someText",  
  "Number": 3.14,  
  "Boolean": true,  
  "Null": null,  
  "Object": {  
    "Key": "Value"  
  },  
  "Array": ["a", "b", "c"]  
}
```

```
const myObj = {  
  String: 'someText',  
  Number: 3.14,  
  Boolean: true,  
  Null: null,  
  Object: {  
    Key: "Value"  
  },  
  Array: ["a", "b", "c"]  
  function: () => {  
    console.log("This is a method")  
  },  
  undefined: undefined  
}
```

```
const json = {  
  "String": "someText",  
  "Number": 3.14,  
  "Boolean": true,  
  "Null": null,  
  "Object": {  
    "Key": "Value"  
  },  
  "Array": ["a", "b", "c"]  
}
```

Exception handling

- Exception handling คือการจัดการข้อผิดพลาด (Error) ที่เกิดขึ้นในโปรแกรมในภาษา JavaScript เป็นวิธีที่จะทำให้โปรแกรมไม่หยุดทำงานและยังสามารถทำงานต่อไปได้



คำสั่ง try catch

26

- คำสั่งที่ใช้สำหรับกำหนดบล็อกเพื่อตรวจสอบความถูกต้องของข้อมูล ในส่วนที่คิดว่าจะเกิดข้อผิดพลาดและจัดการกับข้อผิดพลาดที่อาจเกิดขึ้นในโปรแกรม โดยที่ try จะทำงานก่อน เมื่อไม่มีข้อผิดพลาดจะไม่ทำงานในบล็อกของ catch แต่ถ้าเกิดความผิดพลาด จะเข้ามาทำงานในบล็อกของ catch
 - Try เป็นส่วนที่วางโค้ดที่อาจเกิดข้อผิดพลาดได้
 - Catch เป็นส่วนที่ทำงานเมื่อเกิดข้อผิดพลาดขึ้นโดยจะจัดการกับข้อผิดพลาด

```
try {  
    // Try to do something that may cause errors  
} catch (error) {  
    // Handling errors  
}
```

คำสั่ง throw

27

- ใช้คำสั่ง throw เพื่อสั่งให้โปรแกรมเกิดข้อผิดพลาดขึ้นเองได้ การเกิดข้อผิดพลาดในโปรแกรมจะส่งผลให้โปรแกรมหยุดทำงานถ้าไม่มีตรวจสอบ ข้อผิดพลาดที่เกิดขึ้นจากคำสั่ง throw นั้นสามารถตรวจสอบได้โดยคำสั่ง try catch
- มีรูปแบบการใช้งาน คำสั่ง throw ดังนี้

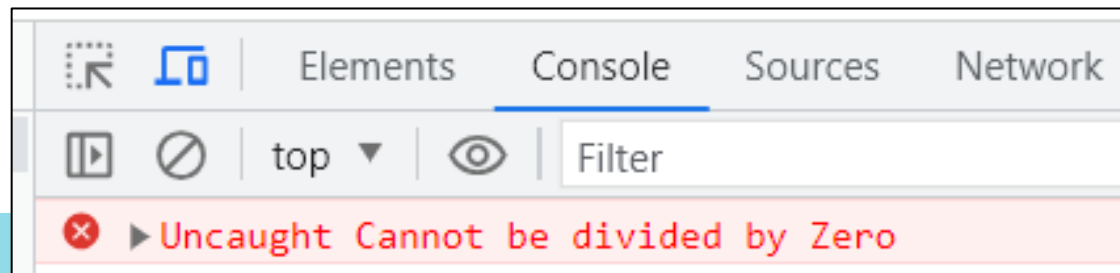
```
throw expression;
```

คำสั่ง throw

28

- ตัวอย่างการใช้ คำสั่ง throw ดังนี้

```
let x = 25;  
let y = 0;  
  
if (y == 0) {  
    throw "Cannot be divided by Zero";  
}  
  
console.log(`${x} / ${y} = ${x / y}`);
```

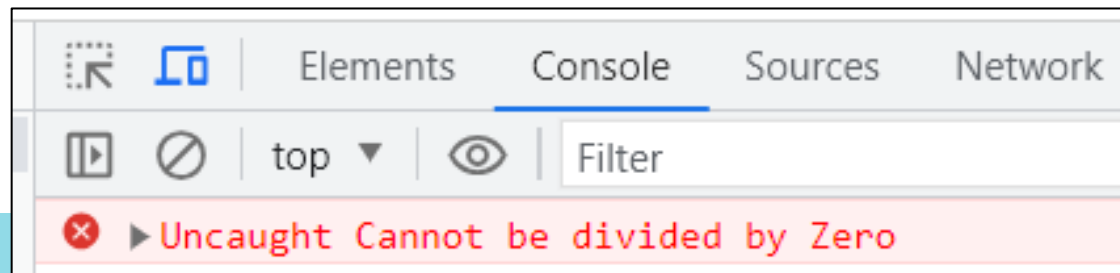


คำสั่ง throw

29

- ตัวอย่างการใช้ คำสั่ง throw ดังนี้

```
let x = 25;  
let y = 0;  
  
if (y == 0) {  
    throw "Cannot be divided by Zero";  
}  
  
console.log(`${x} / ${y} = ${x / y}`);
```



ตัวอย่างการใช้ คำสั่ง throw และ try catch

30

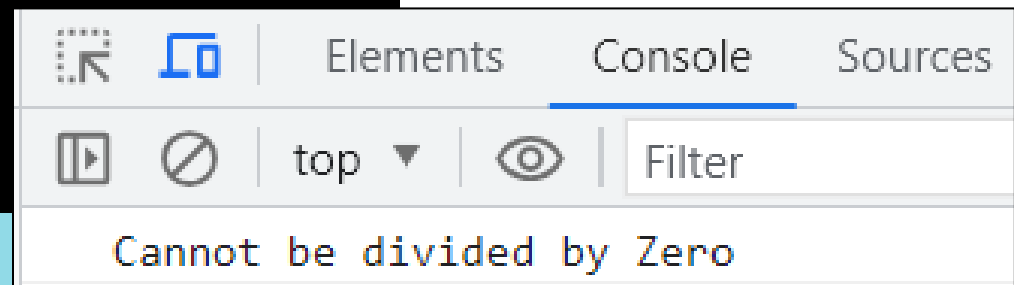
- ตัวอย่างการใช้ คำสั่ง throw และ try catch ดังนี้

```
let x = 25;
let y = 0;

function multiplied_by(a,b) {
  if (b == 0) {
    throw new Error("Cannot be divided by Zero");
  }
  return a/b;
}

try {
  console.log(`${x} / ${y} = ${multiplied_by(x,y)}`);
} catch (error) {
  console.log(error.message);
}
```

ใช้คำสั่ง throw new Exception('ข้อความ')
เพื่อระบุข้อความ หากพบข้อผิดพลาด



คำสั่ง finally

31

- เมื่อการจัดการข้อผิดพลาดจบลง สามารถใช้คำสั่ง finally เพื่อให้โปรแกรมทำงานบางอย่าง
- finally จะทำงานเสมอ และใช้งานร่วมกับคำสั่ง try catch
- มีรูปแบบการใช้งาน finally ดังนี้

```
try {  
    // Do something  
} catch (error) {  
    // Handling error  
} finally {  
    // Always execute this block  
}
```

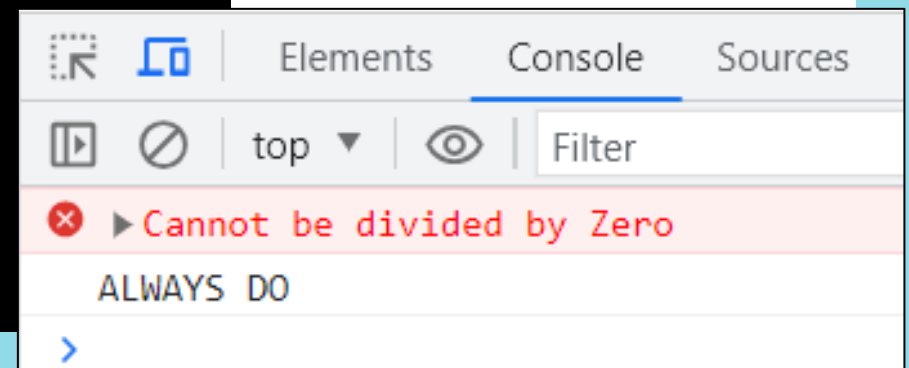
ตัวอย่างการใช้ คำสั่ง throw, try catch และ finally

32

```
let x = 25;
let y = 0;

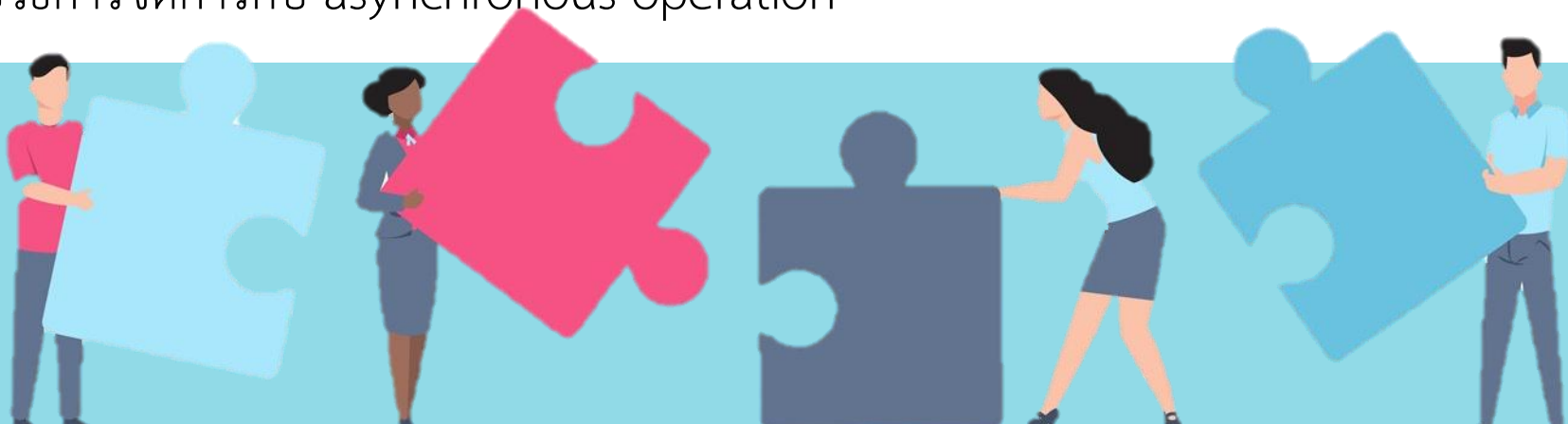
function multiplied_by(a,b) {
  if (b == 0) {
    throw "Cannot be divided by Zero";
  }
  return a/b;
}

try {
  console.log(`${x} / ${y} = ${multiplied_by(x,y)}`);
} catch (error) {
  console.error(`${error}`);
} finally{
  console.log("ALWAYS DO")
}
```



Promise and Asynchronous

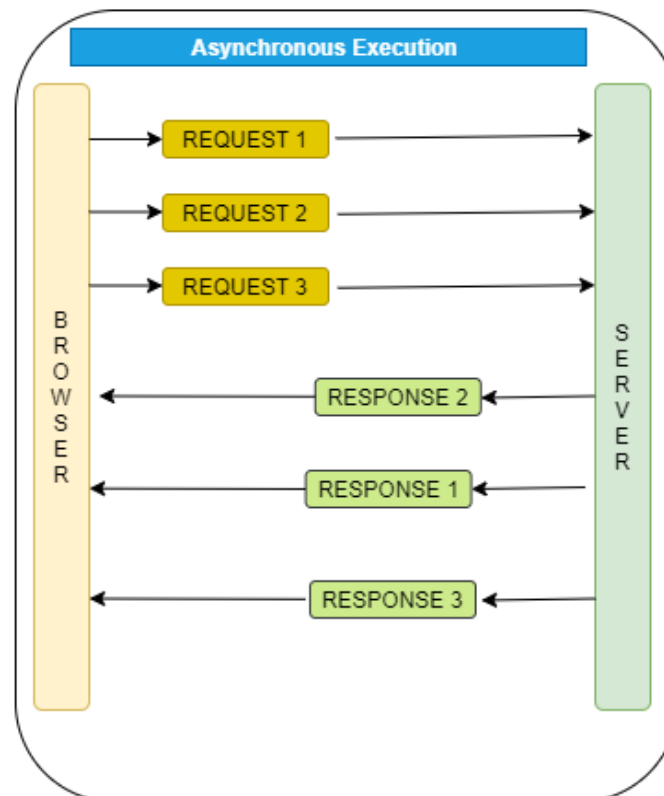
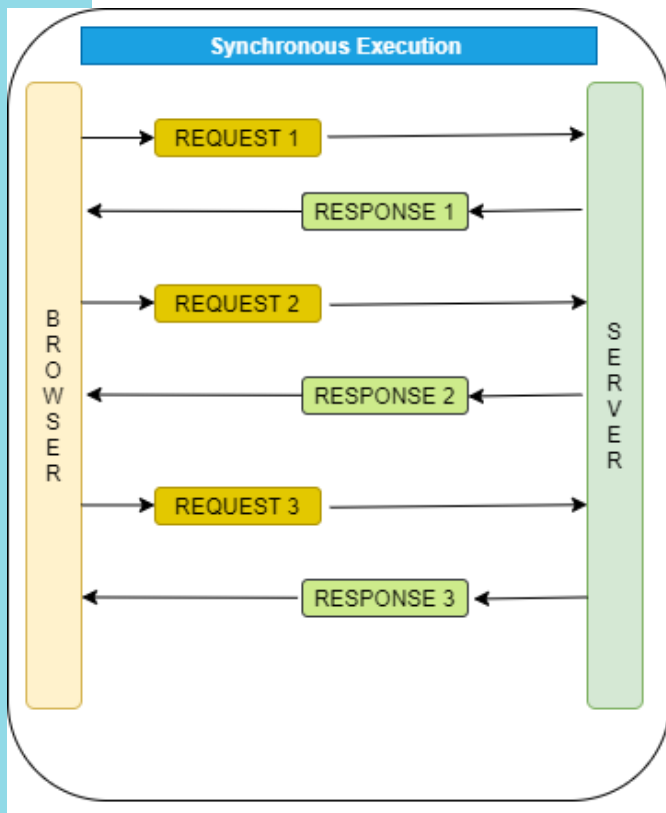
- คำว่า Asynchronous หมายถึงการทำให้โค้ดสามารถรันแยกกันได้
- การทำงานแบบ asynchronous ในภาษา JavaScript คือการทำงานบางอย่างที่ต้องรอผลลัพธ์จากโค้ดที่ใช้เวลานานไปใช้ต่อ
- Promise คือวิธีการจัดการกับ asynchronous operation



Synchronous และ Asynchronous

34

- **Synchronous** การทำงานเป็นลำดับคำสั่งในโปรแกรม แต่ละคำสั่งจะรอให้คำสั่งก่อนหน้าดำเนินการจนเสร็จสิ้น บางครั้งคำสั่งที่สำคัญอาจถูกบล็อกเนื่องจากคำสั่งก่อนหน้าบางคำสั่งทำให้เกิดความล่าช้าในส่วนติดต่อผู้ใช้
- **Asynchronous** คือการทำให้โค้ดสามารถทำงานแยกกันได้ ช่วยให้สามารถดำเนินการคำสั่งถัดไปได้ทันทีที่ไม่รอให้คำสั่งก่อนหน้าเสร็จ และไม่ปิดกั้นการทำงานของคำสั่งเนื่องจากคำสั่งก่อนหน้านี้อยู่

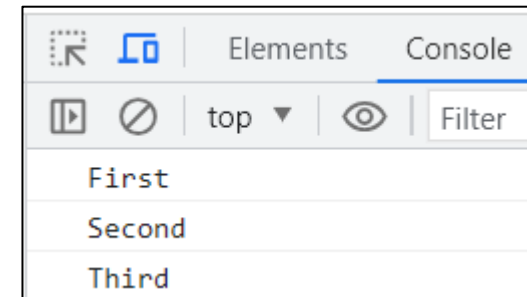


เช่น การ request ข้อมูลจาก server หรือการอ่านข้อมูลจาก database ซึ่งจะใช้เวลาในการทำงานนานกว่าการทำงานที่เป็น synchronous ซึ่งจะต้องรอผลลัพธ์จาก asynchronous operation นั้น ก่อนที่จะทำงานต่อไป

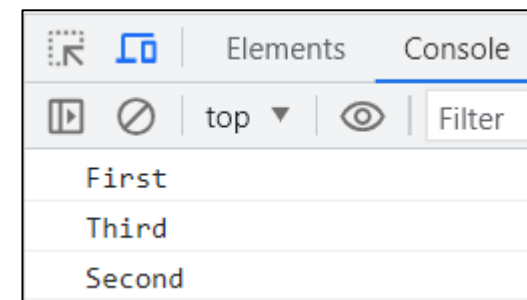
JavaScript มีคำสั่งที่ทำงานแบบ Asynchronous

35

```
console.log('First');  
console.log('Second');  
console.log('Third');
```



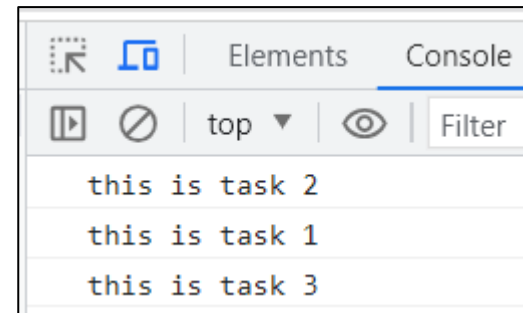
```
console.log('First');  
  
// Set timeout for 2 seconds (1 second = 1000  
milliseconds)  
setTimeout(() => console.log('Second'), 2000);  
  
console.log('Third');
```



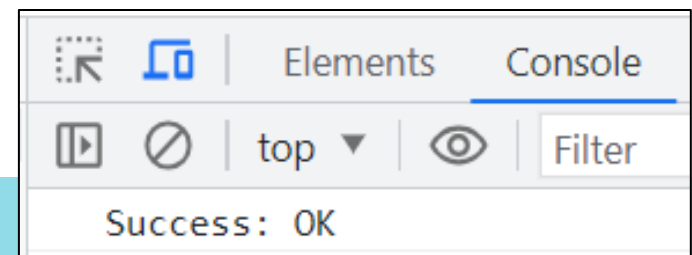
คำสั่ง `setTimeout` ทำให้คำสั่ง `console.log('Second')` ถูกเรียกเข้าไป 2000 ms และด้วยธรรมชาติของ JavaScript มันจะรันคำสั่ง `console.log('Third')` ก่อนโดยไม่รอ แล้วค่อยกลับไปเรียกคำสั่ง `console.log('Second')` ที่หลัง ทำให้ Third ถึง log ก่อน Second

```
function taskOne() {  
  setTimeout(function () {  
    console.log("this is task 1");  
  }, 500);  
}  
function taskTwo() {  
  console.log("this is task 2");  
}  
function taskThree() {  
  setTimeout(function() {  
    console.log("this is task 3");  
  }, 1000)  
}  
  
taskOne();  
taskTwo();  
taskThree();
```

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree ที่แต่ละฟังก์ชันจำลองการใช้เวลาทำงานจนเสร็จต่างกันด้วย setTimeout เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree แต่ผลลัพธ์เป็น this is task 2, this is task 1, this is task 3 เพราะ setTimeout() เป็นคำสั่งที่ทำงานแบบ asynchronous ทำให้ JavaScript จะไล่ทำงานไปตามคำสั่งไปเรื่อยๆ โดยไม่ได้สนใจว่าคำสั่งที่เพิ่งเรียกไปก่อนหน้านี้จะเสร็จหรือไม่ การทำงานแบบนี้จึงอาจเกิดปัญหากับงานที่ต้องการให้ทำตามลำดับได้



- **Promise** คือเป็น object ที่ใช้สำหรับจัดการกับ asynchronous (งานที่ทำเสร็จไม่พร้อมกัน เช่น โหลดไฟล์, เรียก API, อ่านไฟล์ ฯลฯ)
- promise ทำหน้าที่เหมือน “สัญญา” ว่าในอนาคตงานนี้จะ สำเร็จ (fulfilled) หรือ ล้มเหลว (rejected) แล้วสามารถเขียนโค้ดรอผลลัพธ์ได้โดยไม่ต้องหยุดการทำงานของโปรแกรม
- โดยผลลัพธ์ของ promise จะเป็น asynchronous operation และจะมีสถานะที่เป็น fulfilled (operation สำเร็จ) หรือ rejected (operation ล้มเหลว)



- รูปแบบ new Promise((resolve, reject) => {...})

```
let myPromise = new Promise((resolve, reject) => {  
  // ทำงานบางอย่างที่ใช้เวลา เช่น fetch, setTimeout, อ่านไฟล์  
  let success = true;  
  if (success) {  
    resolve("สำเร็จ");  
  } else {  
    reject("ล้มเหลว");  
  }  
});
```

- ใช้งาน → .then(), .catch(), .finally()
 - .then() ใช้สำหรับรับผลลัพธ์ เมื่อ Promise สำเร็จ (fulfilled / resolve) รับค่าที่ถูกส่งออกมาจาก resolve(value)
 - .catch() ใช้สำหรับจัดการ ข้อผิดพลาด (เมื่อ Promise ล้มเหลว / reject) รับค่าที่ถูกส่งออกมาจาก reject(error)
 - ใช้ .finally() (เสริม) ส่วนนี้จะทำงานไม่ว่าจะ success หรือ error

```
let myPromise = new Promise((resolve, reject) => {
  // ทำงานบางอย่างที่ใช้เวลา เช่น fetch, setTimeout, อ่านไฟล์
  let success = true;
  if (success) {
    resolve("สำเร็จ");
  } else {
    reject("ล้มเหลว");
  }
});

myPromise
  .then(result => console.log("ผลลัพธ์:", result)) // ถ้าสำเร็จ
  .catch(error => console.log("เกิดข้อผิดพลาด:", error)) // ถ้าล้มเหลว
  .finally(() => console.log("จบการทำงาน"))
```

Promise Chain (เชื่อมต่อหลาย .then())

40

- Promise Chain คือ Promise ที่เชื่อมต่อหลาย .then() ก็คือการที่เราเอา Promise มาต่อกันหลายขั้นตอน โดยที่ ผลลัพธ์จาก .then() แต่ละตัว จะถูกส่งต่อไปยัง .then() ตัวถัดไป

```
new Promise(resolve => resolve(2))  
  .then(num => num * 2)           // 4  
  .then(num => num + 3)           // 7  
  .then(result => console.log(result)); // 7
```

- สร้างฟังก์ชันเพื่อทำเช็คค่าตัวแปร x

```
let my_Promise = new Promise((fulfill, reject) => {  
  let x = 0; // ลองเปลี่ยนเลขเพื่อดูผลลัพธ์เมื่อสำเร็จและล้มเหลว  
  
  if (x == 0) {  
    fulfill("OK");  
  } else {  
    reject("x must be 0");  
  }  
});  
  
my_Promise  
  .then((value) => console.log("Success: " + value))  
  .catch((value) => console.log("Error: " + value));
```

• my_Promise → คือ **Promise object** ที่คุณสร้างไว้ (เช่น new Promise(...))

• .then(...) → จะทำงาน ถ้า Promise ถูก resolve (สำเร็จ)

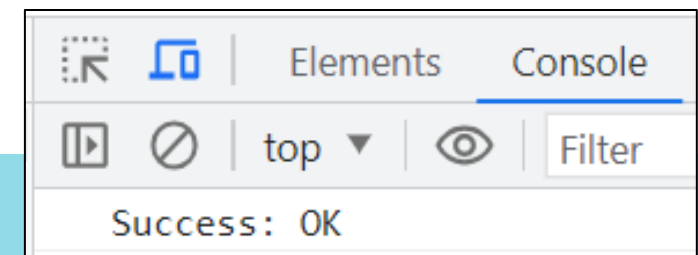
• พารามิเตอร์ (value) = ค่าที่ส่งมาจาก resolve(...)

• .catch(...) → จะทำงาน ถ้า Promise ถูก reject (ล้มเหลว)

• พารามิเตอร์ (value) = ค่าที่ส่งมาจาก reject(...)

จากโค้ดจะเห็นว่าการสร้าง promise ต้องใช้พารามิเตอร์สองตัว คือ fulfill กับ reject โดย fulfill เป็น callback (callback คือ function ที่จะถูกเรียกหลัง function อื่นทำงานเสร็จ) เมื่อทำงานเสร็จ ส่วน reject คือ callback เมื่อเกิด error ขึ้น

โค้ดเป็นการตรวจเช็คค่าของ x ถ้า x มีค่าเท่ากับ 0 promise ทำงานสำเร็จส่งผลลัพธ์เป็น OK แต่ถ้า x มีค่าไม่เท่ากับ 0 promise ทำงานล้มเหลวส่งผลลัพธ์เป็น x must be 0



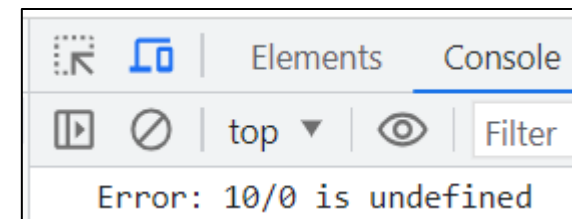
คำสั่ง Promises

42

- สร้างฟังก์ชันเพื่อทำการหาร โดยหารหารตัวเลขที่ใช้หารห้ามเป็นศูนย์จะไม่สามารถหาผลลัพธ์ได้

```
function check(a,b){
  return new Promise(function (fulfill, reject){
    if (b == 0) {
      reject(`${a}/${b} is undefined`);
    } else {
      fulfill(`${a}/${b}=${a/b}`);
    }
  });
}

let ans = check(10,0) //ลองเปลี่ยนเลขเพื่อดูผลลัพธ์เมื่อสำเร็จและล้มเหลว
ans.then((value) => console.log("Success: " + value))
.catch((value) => console.log("Error: " + value));
```



```
console.log('First');

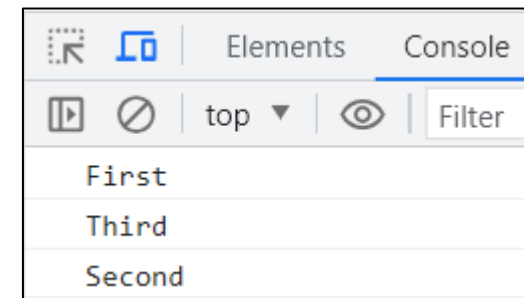
const promise = new Promise((fulfill, reject) => {
  // Set timeout for 2 seconds
  setTimeout(() => fulfill('Second'), 2000)
});

promise.then(value =>{
  console.log(`${value}`)
})

console.log('Third');
```

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree ที่แต่ละฟังก์ชันจำลองการใช้เวลาทำงานจนเสร็จต่างกันด้วย setTimeout

เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree และฟังก์ชันแสดงผลลัพธ์ไม่ตรงตามลำดับ



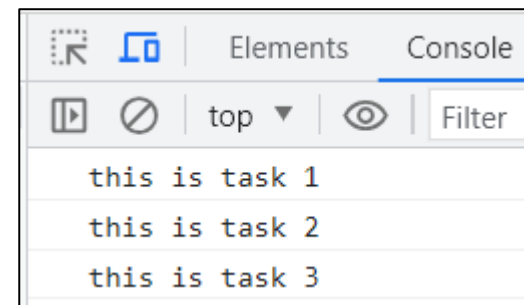
```
function taskOne() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}
function taskTwo() {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}
function taskThree() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}
```

```
taskOne().then(value =>{console.log(`${value}`)})
taskTwo().then(value =>{console.log(`${value}`)})
taskThree().then(value =>{console.log(`${value}`)}) });
});
});
```

คำสั่ง Promises

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree (เขียนฟังก์ชันแบบปกติ) ที่แต่ละฟังก์ชันจำลองการใช้เวลาทำงานจนเสร็จต่างกันด้วย setTimeout

เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree และฟังก์ชันต้องทำงานตามลำดับ



โค้ดด้านบนนี้ที่ใช้ Promise จะเห็นว่าโค้ดสามารถทำงานเรียงลำดับถูกต้อง แต่โค้ดที่ได้ก็เริ่มจะดูอ่านยากขึ้นตามไปด้วย เพราะ Promise จะต้องเขียน .then() ทุกครั้งที่ต้องการทำคำสั่งถัด ดังนั้น async/await จึงนำมาใช้จัดการ Promise ให้ดูอ่านง่ายขึ้น

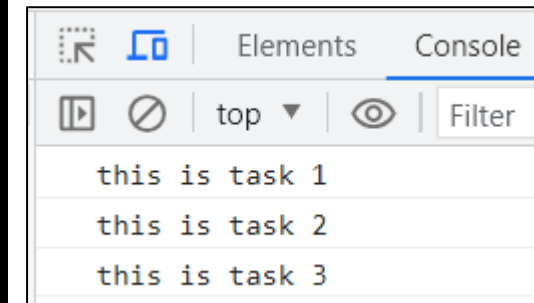
คำสั่ง Promises

```
const taskOne =()=> {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}
const taskTwo=()=> {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}
const taskThree=()=> {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}
```

```
taskOne().then(value =>{console.log(`${value}`)})
taskTwo().then(value =>{console.log(`${value}`)})
taskThree().then(value =>{console.log(`${value}`)})
});
});
```

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree (เขียนแบบ arrow function) ที่แต่ละฟังก์ชันจำลองการใช้เวลาทำงานจนเสร็จต่างกันด้วย setTimeout

เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree และฟังก์ชันต้องทำงานตามลำดับ



โค้ดด้านบนนี้ที่ใช้ Promise จะเห็นว่าโค้ดสามารถทำงานเรียงลำดับถูกต้อง แต่โค้ดที่ได้ก็เริ่มจะดูอ่านยากขึ้นตามไปด้วย เพราะ Promise จะต้องเขียน .then() ทุกครั้งที่ต้องการทำคำสั่งถัด ดังนั้น async/await จึงนำมาใช้จัดการ Promise ให้ดูอ่านง่ายขึ้น

```

const taskOne =()=> {
  return new Promise(function (resolve, reject){
    let x=10
    resolve(x)
  });
}
const taskTwo=(x)=> {
  return new Promise(function (resolve, reject){
    let y=x+5
    resolve(y)
  });
}
const taskThree=(y)=> {
  return new Promise(function (resolve, reject){
    let z=y+5
    resolve(z)
  });
}

taskOne().then((x) =>{
  console.log(`x=${x}`)
  return taskTwo(x)
}).then((y) =>{
  console.log(`y=${y}`)
  return taskThree(y)
}).then((z) =>{
  console.log(`z=${z}`)
}).catch((error) =>{
  console.log(`${error}`)
})

```

คำสั่ง Promises

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree (เขียนแบบ arrow function) ที่แต่ละฟังก์ชันจะต้องมีการทำงานเป็นลำดับเพื่อส่งค่าให้กับฟังก์ชันถัดไป ถ้าฟังก์ชันใดไม่สามารถทำงานได้ฟังก์ชันถัดไปก็ไม่สามารถทำงานได้เช่นกัน

เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree และ ฟังก์ชันต้องทำงานตามลำดับ

```

x=10
y=15
z=20

```

โค้ดด้านบนนี้ที่ใช้ Promise จะเห็นว่าโค้ดสามารถทำงานเรียงลำดับถูกต้อง แต่โค้ดที่ได้ก็เริ่มจะดูอ่านยากขึ้นตามไปด้วย เพราะ Promise จะต้องเขียน .then() ทุกครั้งที่ต้องการทำคำสั่งถัด ดังนั้น async/await จึงนำมาใช้จัดการ Promise ให้ดูอ่านง่ายขึ้น

คำสั่ง Async/Await

- *async/await* เครื่องมือสำหรับการจัดการกับการทำงานแบบ asynchronous ในภาษา JavaScript *ใช้กับการทำงานบางอย่างที่ต้องรอผลลัพธ์*
- *async, await* เป็นตัวช่วยจัดการ *Promise* ให้ดูอ่านง่ายขึ้น และดูเหมือนโค้ด synchronous (ทำทีละบรรทัด)
- จะเขียน *async* ไว้หน้า *function* (เท่านั้น) จะเป็นการระบุว่า *function* นั้นทำงานแบบ *asynchronous*
 - *async* → ใช้ประกาศว่าฟังก์ชันนี้จะ *return Promise*
- จะเติม *await* เข้าไปข้างหน้าก่อนการเรียก *function* นั้นเพื่อบอกให้ JavaScript รอจนกว่าคำสั่งนั้นจะเสร็จ ถึงค่อยไปทำงานต่อไป
- ฟังก์ชันที่จะมี *await* อยู่ข้างในได้ต้องประกาศเป็น *async* เสมอ
- นั่นคือแต่คำสั่ง *await* มีข้อจำกัดคือจะใช้ได้แค่เฉพาะในฟังก์ชัน *async* เท่านั้น
- โดยการเรียก *await* นั้นจะสามารถใช้กับ *function* ได้ 2 แบบก็คือ *function* ที่ *return* ค่า *Promise object* และอีกแบบหนึ่งคือ *async function*
- *สามารถใช้ async/await กับ API ที่ใช้ promise ได้*

```

function taskOne() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}

function taskTwo() {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}

function taskThree() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}

async function asyncCall(){
  const task1 = await taskOne()
  console.log(task1)

  const task2 = await taskTwo()
  console.log(task2)

  const task3 = await taskThree()
  console.log(task3)
}

asyncCall();

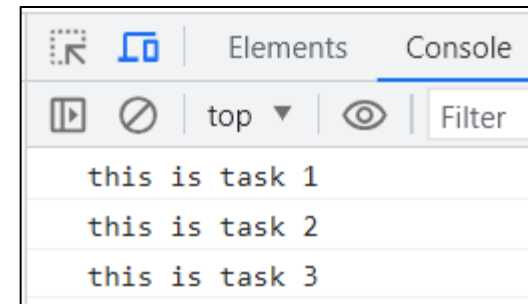
```

คำสั่ง Async/Await

- ตัวอย่างการใช้ await ใช้กับ function ที่ return ค่า Promise object

มีฟังก์ชัน 3 ฟังก์ชัน คือ taskOne, taskTwo และ taskThree ที่แต่ละฟังก์ชันจำลองการใช้เวลาทำงานจนเสร็จต่างกันด้วย setTimeout

เมื่อเรียกใช้ฟังก์ชันให้ทำงานเรียงลำดับ taskOne, taskTwo, taskThree และฟังก์ชันต้องทำงานตามลำดับ



```

function taskOne() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}
function taskTwo() {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}
function taskThree() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}

async function asyncCall() {
  try {
    const task1 = await taskOne();
    console.log(task1);

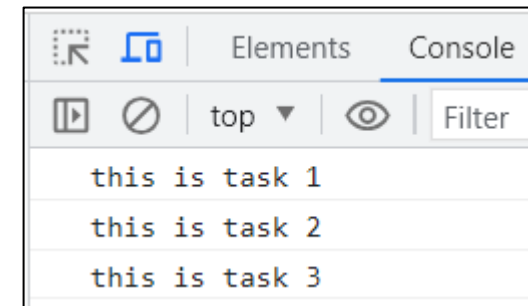
    const task2 = await taskTwo();
    console.log(task2);

    const task3 = await taskThree();
    console.log(task3);
  } catch (err) {
    console.error("Error:", err);
  }
}
asyncCall();

```

คำสั่ง Async/Await

- เวลาใช้ await กับฟังก์ชันที่ return Promise
 - ถ้า Promise resolve → โค้ดจะทำงานต่อไปปกติ
 - ถ้า Promise reject → มันจะ “โยน error” ออกมาเหมือน throw
- ดังนั้นถ้าไม่มี try...catch โปรแกรมอาจหยุดด้วย error ที่ไม่ได้ถูกจัดการ



```

function taskOne() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}
function taskTwo() {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}
function taskThree() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}

async function asyncCall() {
  try {
    const [t1, t2, t3] = await Promise.all([
      taskOne(),
      taskTwo(),
      taskThree()
    ]);

    console.log(t1);
    console.log(t2);
    console.log(t3);

  } catch (err) {
    console.error("เกิดข้อผิดพลาด:", err);
  }
}

asyncCall();

```

คำสั่ง Async/Await

- `Promise.all()` → จะ รอจนกว่าทุกตัวจะ fulfilled (สำเร็จ) หรือถ้ามี ตัวใด reject ก่อน ทั้งหมดจะ reject ทันที ลำดับของผลลัพธ์จะตรงกับลำดับที่ใส่ไว้ใน array ไม่ใช่ลำดับที่เสร็จจริง

คำสั่ง Async/Await

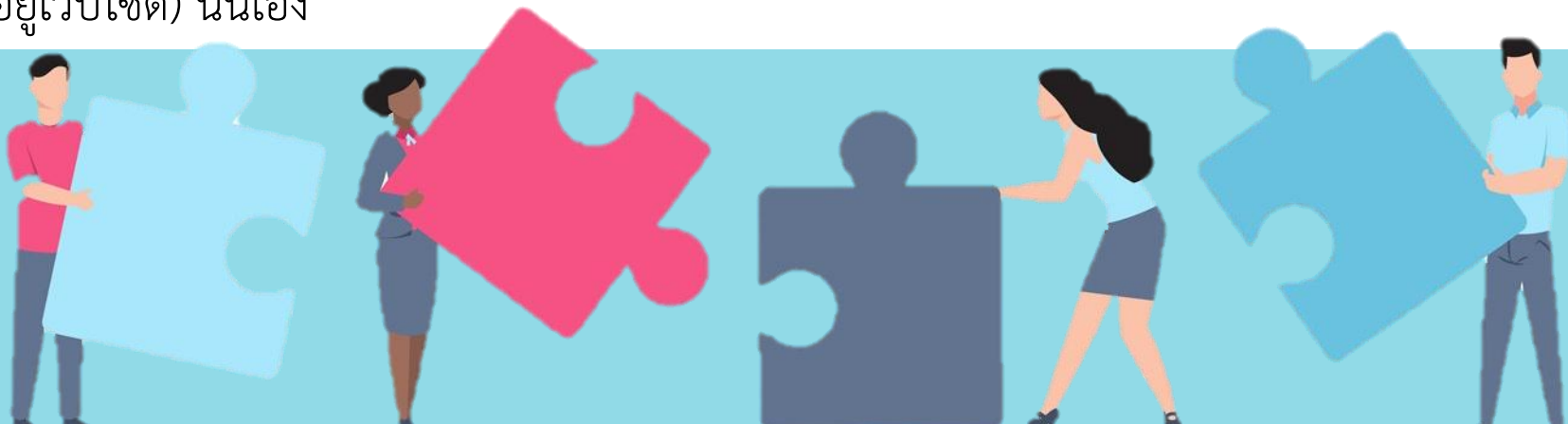
```
function taskOne() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 1'), 500)
  });
}
function taskTwo() {
  return new Promise(function (resolve, reject){
    resolve('this is task 2')
  });
}
function taskThree() {
  return new Promise(function (resolve, reject){
    setTimeout(() => resolve('this is task 3'), 1000)
  });
}

async function asyncCall() {
  try {
    const winner = await Promise.race([taskOne(), taskTwo(),
taskThree()]);
    console.log(winner); // "this is task 2"
  } catch (err) {
    console.error("เกิดข้อผิดพลาด:", err);
  }
}
asyncCall();
```

- Promise.race() → เอาตัวที่เสร็จก่อน
- คืนค่า ผลเดียว (ตัวที่ “เสร็จก่อน” ไม่ว่าจะสำเร็จหรือพัง)

URL Component

- ใช้ URL เพื่อเข้าเว็บไซต์ ผ่านเว็บเบราว์เซอร์ (Web Browser)
- คำว่า "URL" ย่อมาจากคำว่า "Uniform Resource Locator"
- หมายถึงตัวระบุที่อยู่ (Address) ของข้อมูลต่าง ๆ ภายในอินเทอร์เน็ตที่ทำให้เราสามารถเข้าถึงไฟล์ (File) หรือหน้าเว็บไซต์ (Webpage) ต่าง ๆ ได้อย่างถูกต้อง (คล้ายกับ "เลขที่บ้าน" ที่ทำให้พนักงานส่งของหาบ้านเราเจอเวลาสั่งซื้อของออนไลน์) ซึ่งส่วนใหญ่แล้วเรามักคุ้นเคยกับ URL ของเว็บไซต์มากกว่า จึงทำให้บางคนนิยมเรียก URL ว่า "Web Address" (ที่อยู่เว็บไซต์) นั่นเอง



- **Protocol** คือ ข้อกำหนดซึ่งประกอบด้วยกฎหรือภาษาสื่อสารที่ใช้เป็น ภาษากลางในการสื่อสารระหว่างคอมพิวเตอร์ด้วยกัน
 - การที่เครื่องคอมพิวเตอร์ที่ถูกเชื่อมโยงกันไว้ในระบบจะสามารถติดต่อสื่อสารกันได้นั้น จำเป็นจะต้องมีการสื่อสารที่เรียกว่า โปรโตคอล (Protocol) โปรโตคอลช่วยให้ระบบคอมพิวเตอร์สองระบบ ที่แตกต่างกันสามารถสื่อสารกันอย่างเข้าใจได้ คือ ข้อตกลงที่กำหนด เกี่ยวกับการสื่อสารระหว่างเครื่องคอมพิวเตอร์ต่างๆ ทั้งวิธีการส่งและรับข้อมูล วิธีการตรวจสอบข้อผิดพลาดของการส่งและรับข้อมูล การแสดงผลข้อมูลเมื่อส่งและรับกันระหว่างเครื่องสองเครื่อง ดังนั้นจะเห็นได้ว่า โปรโตคอลมีความสำคัญมากในการสื่อสารบนเครือข่าย หากไม่มีโปรโตคอลแล้ว การสื่อสารบนเครือข่ายจะไม่สามารถเกิดขึ้นได้
- **ประเภทของ Network Protocols**
 - Communication Protocol (โปรโตคอลด้านการสื่อสาร) : ใช้ในการแลกเปลี่ยนข้อมูลขั้นพื้นฐาน เช่น TCP/IP และ HTTP
 - Management Protocol (โปรโตคอลด้านการจัดการ) : มีหน้าที่ดูแลรักษา และบริหารเครือข่ายผ่านโปรโตคอล เช่น ICMP และ SNMP
 - Security Protocol (โปรโตคอลด้านความปลอดภัย) : ช่วยปกป้องข้อมูลจากการถูกโจมตีจากอาชญากรไซเบอร์ เช่น HTTPS, SFTP และ SSL

- **DNS (Domain Name System)** หรือที่บางคนนิยมเรียกว่า ชื่อโฮสต์ (Host Name) ที่เป็นองค์ประกอบสำคัญของ URL ที่ช่วยให้ผู้ใช้สามารถเข้าถึงเว็บไซต์ที่ต้องการได้อย่างถูกต้อง (คล้ายกับ "ชื่อผู้รับ" ที่ช่วยให้พนักงานส่งของสามารถส่งมอบของได้ถูกคน) ซึ่ง DNS ของเว็บไซต์แต่ละเว็บจะมีความ "แตกต่างกัน" กันออกไปตามที่คุณพัฒนานำหนด แต่ "ส่วนท้าย (นามสกุล)" ของ DNS จะมีการกำกับตามประเภทของเว็บไซต์นั้น ๆ เช่น
 - .com (บริษัท/องค์กร)
 - .co.th (บริษัทที่ทำการค้าในประเทศไทย)
 - .org (องค์กรไม่แสวงผลกำไร)
 - .ac.th (สถานศึกษาที่ตั้งอยู่ในประเทศไทย)
 - .net (เครือข่าย)

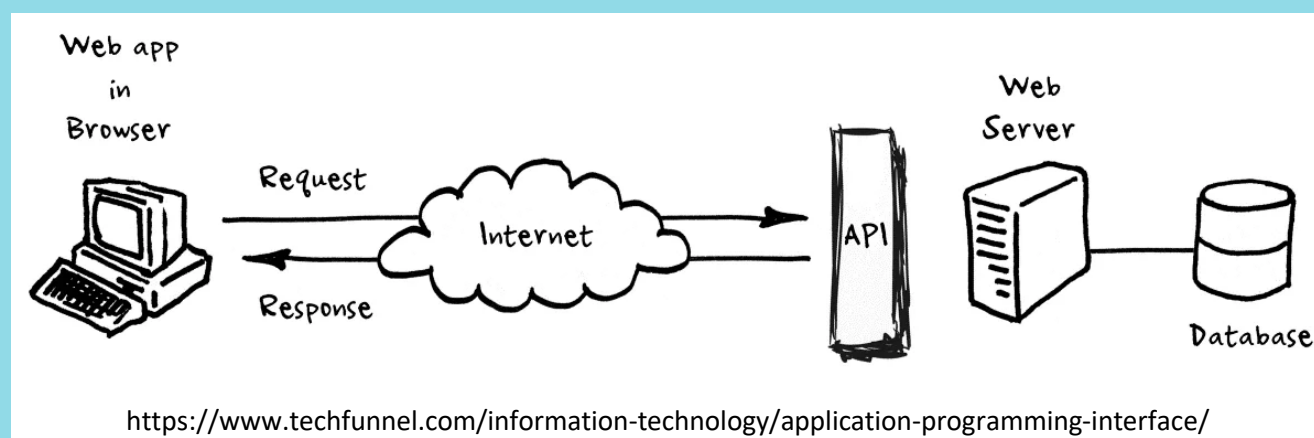
URL ประกอบไปด้วยอะไรบ้าง ?

55

<https://www.example.com/path/to/resource?name=value#section1>



- **Scheme** Protocol Identifier หรือ Scheme จะอยู่บริเวณ "ด้านหน้า" ของ URL เป็นส่วนที่ทำให้สามารถเข้าถึง "ข้อมูล" ต่าง ๆ ผ่าน DNS (Domain Name System) โดย Protocol Identifier (Scheme) ที่คาดทุกคนน่าจะคุ้นเคยกันดีก็ได้แก่ http, https และ ftps เป็นต้น
- **Host** ชื่อโดเมนหรือที่อยู่ IP เพื่อให้ผู้ใช้งานสามารถเข้าถึงเว็บไซต์ได้
- **Path** เส้นทางที่สามารถเข้าถึงหน้าของเว็บไซต์ที่ต้องการ โดยแต่ละหน้าจะมี Path ที่แตกต่างกัน
- **Query** ข้อมูลที่ส่งผ่านไปยังเว็บไซต์ต่าง ๆ มักปรากฏเครื่องหมาย "?" ตามด้วยค่า Parameter ซึ่งหากมี Parameter มากกว่า 1 ค่าจะแสดงค่าเป็นคู่ที่ใช้เครื่องหมาย "&" คั่นระหว่างค่า และแบ่งคู่ด้วยเครื่องหมาย "="
- **Fragment** ส่วนของ URL ที่ระบุจุดที่ปักหมุดเอาไว้ภายในหน้าเว็บไซต์ ขึ้นต้นด้วย "#" เสมอ โดยบนเว็บไซต์ก็จะกระโดดไปยัง "วินาที" ที่มาร์คจุดเอาไว้ ส่วนในหน้าเว็บไซต์ก็จะข้ามไปที่ "หัวข้อ" ที่ผู้ใช้ต้องการ



API

- API ย่อมาจาก Application Programming Interface
- ช่องทางการเชื่อมต่อที่ทาง Server API ที่ให้เชื่อมต่อ เพื่อดึงข้อมูลผ่าน URL
- ใช้เป็นตัวกลางในการเชื่อมต่อกับโปรแกรมต่อโปรแกรม หรือ เชื่อมต่อกับระบบปฏิบัติการ เข้าด้วยกัน
- เป็นตัวช่วยในการให้เข้าถึงการรับ หรือการส่งข้อมูลได้ง่ายขึ้น

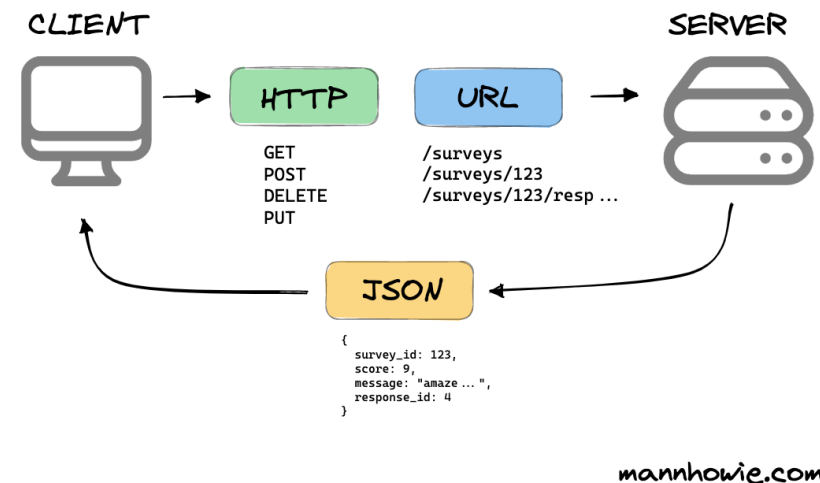


REST, RESTful API

- REST ย่อมาจาก Represent State Transfer: Web Service สำหรับการติดต่อกันทาง Internet โดยอาศัย URI/URL
- โดยการสื่อสารจะสามารถใช้ได้หลายรูปแบบ XML, HTML และ JSON

REST คือ สถาปัตยกรรม
RESTful คือ การนำ REST มาใช้

WHAT IS A REST API?

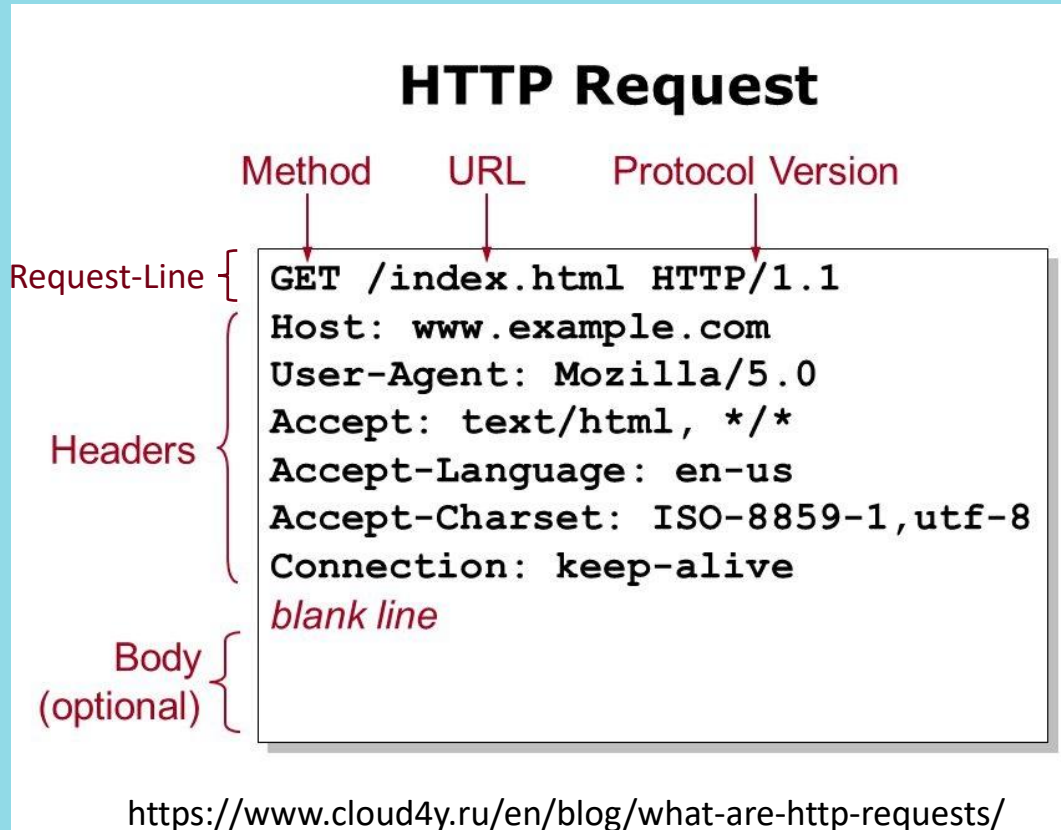


1. ไคลเอ็นต์ส่งคำขอไปยังเซิร์ฟเวอร์ ไคลเอ็นต์ปฏิบัติตามเอกสารประกอบ API เพื่อจัดรูปแบบคำขอในลักษณะที่เซิร์ฟเวอร์เข้าใจได้
2. เซิร์ฟเวอร์รับรองความถูกต้องของไคลเอ็นต์ และยืนยันว่าไคลเอ็นต์มีสิทธิ์ส่งคำขอดังกล่าว
3. เซิร์ฟเวอร์รับคำขอและประมวลผลเป็นการภายใน
4. เซิร์ฟเวอร์ส่งคืนการตอบสนองกลับไปยังไคลเอ็นต์ การตอบสนองมีข้อมูลที่บอกให้ลูกค้าทราบว่าคำขอดังกล่าวสำเร็จหรือไม่ การตอบสนองยังรวมถึงข้อมูลใดๆ ที่ไคลเอ็นต์ร้องขออีกด้วย

ติดต่อกันระหว่างหน้าเว็บไซต์ และ ส่วนของการจัดการฐานข้อมูล

58

- หน้าเว็บไซต์ และ ส่วนของการจัดการฐานข้อมูล จะต้องมีการทำงานร่วมกันโดยติดต่อสื่อสารกันและกันในการรับ-ส่ง ข้อมูล เส้นทางที่ 2 ส่วนนี้ติดต่อกันเรียกว่า HTTP request โดย เว็บไซต์ จะทำการร้องขอข้อมูล (request) และ ส่วนของการจัดการฐานข้อมูลจะเป็นส่วนที่ทำการส่งข้อมูลกลับ (response)
- เส้นทาง HTTP Request ที่มีการรับ-ส่ง จะใช้คำสั่ง หรือ Method ตัวหลักๆ ดังนี้
 - GET คือ ร้องขอข้อมูลจาก เซิร์ฟเวอร์
 - POST คือ ร้องขอเพิ่มข้อมูลไปที่เซิร์ฟเวอร์เพื่อเพิ่มฐานข้อมูล
 - PUT คือ ร้องขอแก้ไขข้อมูลไปที่เซิร์ฟเวอร์เพื่อแก้ไขฐานข้อมูล
 - DELETE คือ ร้องขอการลบข้อมูลไปที่เซิร์ฟเวอร์เพื่อลบฐานข้อมูล



- HTTP request คือ “คำขอ” ที่ฝั่งไคลเอนต์ (เช่น เบราว์เซอร์/แอป) ส่งไปยังเซิร์ฟเวอร์ผ่านโปรโตคอล HTTP/HTTPS เพื่อขอข้อมูลหรือสั่งให้ทำงานบางอย่าง (เช่น ดึงรายการสินค้า, สร้างผู้ใช้ ฯลฯ)
- ส่วนประกอบสำคัญ
- **Method:** วิธีการของคำขอ – GET (ขอข้อมูล), POST (ส่ง/สร้าง), PUT (แทนที่), PATCH (แก้บางส่วน), DELETE (ลบ) , HEAD (ทำงานเหมือน GET แต่ไม่ส่ง body กลับมา), OPTIONS (ขอให้เซิร์ฟเวอร์บอกว่า ทรัพยากรนี้รองรับวิธีใดบ้าง (เช่น GET, POST, PUT ...))
- **URL:** ปลายทางของทรัพยากร เช่น /api/users?role=admin
- **Headers:** เมตาดาต้า เช่น Content-Type (ระบุ ชนิดของข้อมูลในบอดีที่ส่งไป), Authorization (แนบข้อมูลยืนยันตัวตน/สิทธิ์), Accept (บอกว่าอยากได้คำตอบรูปแบบไหน), Cookie (ส่งคุกกี้ของโดเมนไปให้เซิร์ฟเวอร์)
- **Body (ถ้ามี):** เนื้อข้อมูลที่ส่งไป เช่น JSON, form-data, ไฟล์

HTTP Request

60

- ตัวอย่าง API ที่นำมาใช้งานในตัวอย่าง: <https://rickandmortyapi.com/api/character>

The screenshot displays a web browser window at rickandmortyapi.com/api/character. The page content shows a JSON response for the character Rick Sanchez. The Network tab in Chrome DevTools is open, showing a GET request to the same URL. The request status is 304 Not Modified, and the response headers are visible.

JSON Response:

```
{
  "info": {
    "count": 826,
    "pages": 42,
    "next": "https://rickandmortyapi.com/api/character?page=2",
    "prev": null
  },
  "results": [
    {
      "id": 1,
      "name": "Rick Sanchez",
      "status": "Alive",
      "species": "Human",
      "type": "",
      "gender": "Male",
      "origin": {
        "name": "Earth (C-137)",
        "url": "https://rickandmortyapi.com/api/location/1"
      },
      "location": {
        "name": "Citadel of Ricks",
        "url": "https://rickandmortyapi.com/api/location/3"
      },
      "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
      "episode": [
        "https://rickandmortyapi.com/api/episode/1",
        "https://rickandmortyapi.com/api/episode/2",
        "https://rickandmortyapi.com/api/episode/3",
        "https://rickandmortyapi.com/api/episode/4",
        "https://rickandmortyapi.com/api/episode/5",
        "https://rickandmortyapi.com/api/episode/6",
        "https://rickandmortyapi.com/api/episode/7",
        "https://rickandmortyapi.com/api/episode/8",
        "https://rickandmortyapi.com/api/episode/9",
        "https://rickandmortyapi.com/api/episode/10",
        "https://rickandmortyapi.com/api/episode/11",
        "https://rickandmortyapi.com/api/episode/12",
        "https://rickandmortyapi.com/api/episode/13",
        "https://rickandmortyapi.com/api/episode/14",
        "https://rickandmortyapi.com/api/episode/15",
        "https://rickandmortyapi.com/api/episode/16",
        "https://rickandmortyapi.com/api/episode/17",
        "https://rickandmortyapi.com/api/episode/18",
        "https://rickandmortyapi.com/api/episode/19",
        "https://rickandmortyapi.com/api/episode/20",
        "https://rickandmortyapi.com/api/episode/21",
        "https://rickandmortyapi.com/api/episode/22",
        "https://rickandmortyapi.com/api/episode/23",
        "https://rickandmortyapi.com/api/episode/24",
        "https://rickandmortyapi.com/api/episode/25",
        "https://rickandmortyapi.com/api/episode/26",
        "https://rickandmortyapi.com/api/episode/27"
      ]
    }
  ]
}
```

Network Tab Details:

- Name:** character
- General:**
 - Request URL: <https://rickandmortyapi.com/api/character>
 - Request Method: GET
 - Status Code: 304 Not Modified
 - Remote Address: 52.74.166.77:443
 - Referrer Policy: strict-origin-when-cross-origin
- Response Headers:**
 - Cache-Control: max-age=259200
 - Cache-Status: "Netlify Edge"; hit
 - Date: Wed, 21 Aug 2024 12:18:18 GMT
 - Etag: W/"4c28-8Xn6OSgPYZqE/r+q8L2TeK0Efo"
 - Expires: Sat, 24 Aug 2024 03:04:17 GMT
 - Server: Netlify
 - Strict-Transport-Security: max-age=31536000
 - Vary: Accept-Encoding
 - X-Nf-Request-Id: 01J5TDZ839S1K4DNH4T50G7B8G
- Request Headers:** (Empty)

HTTP-Response

Protocol Status Status
Version code message

HTTP/1.1 200 OK

HTTP response
status line

Date: Wed, 06 Jul 2022 09:30:28 GMT
Accept-Ranges: bytes
Content-Length: 2005
Content-Type: text/css; charset=UTF-8
<CRLF>

HTTP response
headers

```
nav.navbar {  
  ...some style  
}
```

HTTP response
body

<https://softuni.org/dev-concepts/everything-you-need-to-know-about-http-protocol/>

- การ Response จะเกิดขึ้นหลังจากส่ง Request ไปที่ server แล้ว server ก็จะมีการตอบรับกลับมาซึ่งจะเรียกข้อความที่ตอบกลับว่า http-Response message โดย Response message จะประกอบไปด้วย 3 ส่วนหลักคือ
- เพื่อดูว่า status-code มีอะไรบ้างและความหมายอย่างไร
 - 2xx (สำเร็จ) หมายความว่า request นั้นได้รับแล้วและกระทำตาม method สำเร็จโดย Server
 - 200 Ok เป็นมาตรฐานของ HTTP Response นั้น Success สำหรับ GET, PUT หรือ POST
 - 201 Create เป็น Response สำหรับข้อมูลใหม่ได้ถูกสร้างขึ้น ใช้สำหรับ POST
 - 204 No Content เป็น Response สำหรับ request ที่ดำเนินการ Success แต่ไม่ได้ return ข้อมูลกลับ
 - 3xx (Redirection)
 - 304 Not Modified เป็น status code ที่บอกว่า client ได้รับการ Response แล้วอยู่ใน cache และไม่จำเป็นต้องส่งผ่านข้อมูลเดิมอีกครั้ง
 - 4xx (Client error) โดย status ในกลุ่มนี้จะบอก client ว่า request ที่เข้ามา error
 - 400 Bad Request บอกว่า request ที่ส่งมาไม่ถูกดำเนินการ เนื่องจากส่งข้อมูลไปไม่ตรงกับที่กำหนดไว้
 - 401 Unauthorized บอกว่า client ไม่ได้รับอนุญาตในการเข้าถึงข้อมูลและควรส่ง credential มาพร้อม request
 - 403 Forbidden บ่งบอกว่า request นั้นถูกต้องและ client ได้รับการอนุญาต แต่ Client ไม่ได้รับการอนุญาตให้เข้าถึงข้อมูลด้วยเหตุผลบางประการ
 - 404 Not Found บ่งบอกว่า resource ที่ request มานั้น ไม่มีในระบบ
 - 405 Method Not Allowed บอกว่า request method ที่เข้ามาไม่ถูกต้องกับที่ gateway กำหนด ให้เรียกด้วย method อื่น
 - 410 Gone บ่งบอกว่าข้อมูลที่ต้องการนั้นไม่มีอยู่แล้ว หรืออาจจะย้ายไปที่อื่น
 - 5xx (Server error)
 - 500 Internal Server Error บอกว่า request ถูกต้อง แต่มีความผิดพลาดเกิดขึ้นกับการทำงาน
 - 502 Bad Gateway บอกว่า gateway ได้รับ error จาก server
 - 503 Service Unavailable บอกว่า server ใช้การไม่ได้ โดยส่วนใหญ่แล้วจะอยู่ในช่วง maintain
 - 504 Gateway Timeout หมายถึงระยะเวลาในการเชื่อมต่อ gateway เกินที่กำหนด

<https://medium.com/icreativesystems/basic-http-3a2b05e5aa19>

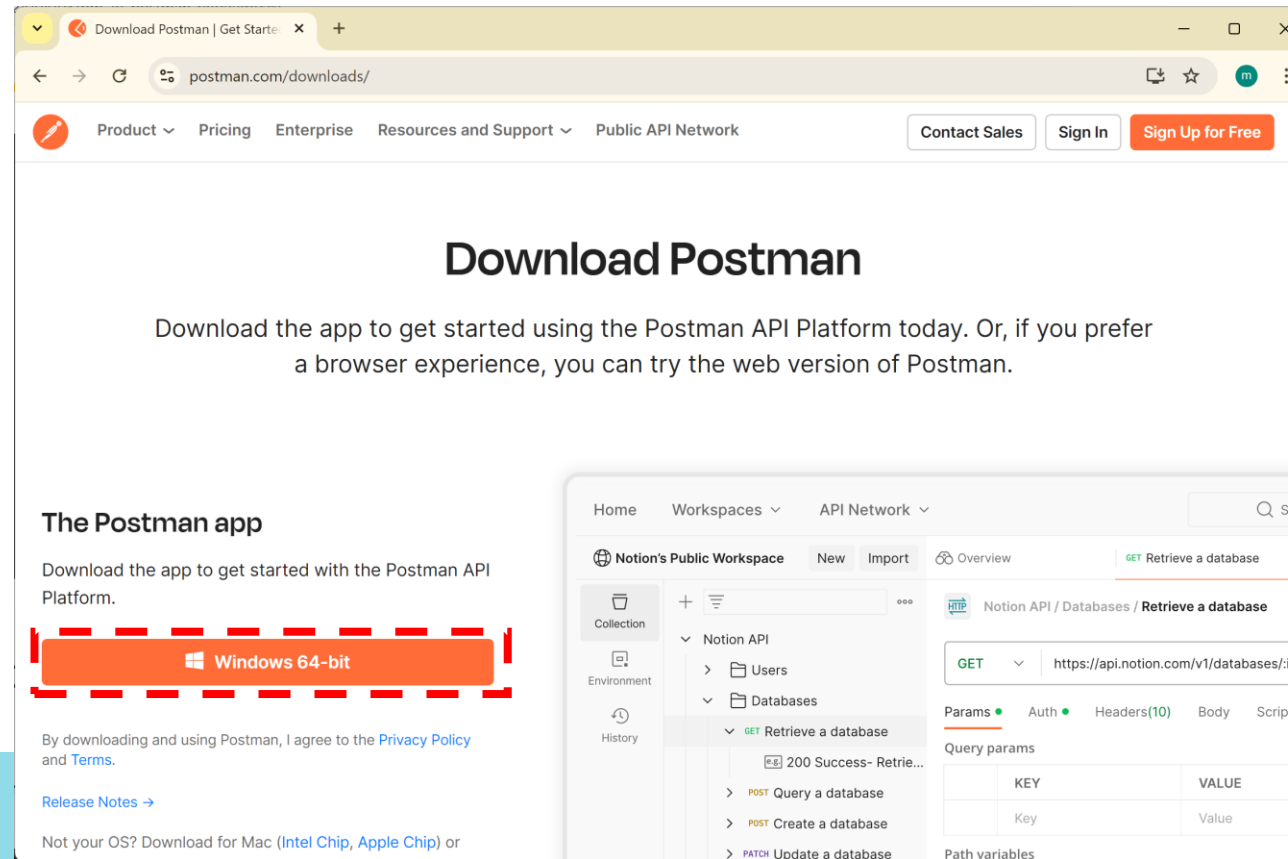


POSTMAN

Postman

62

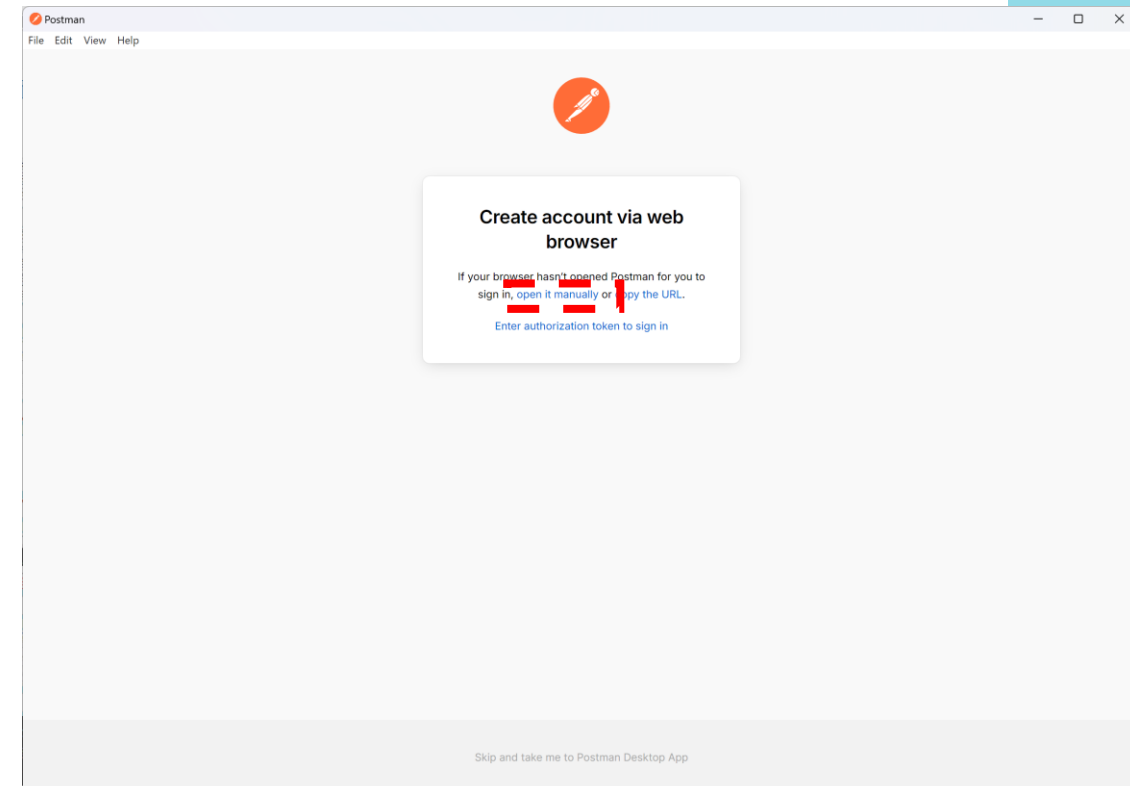
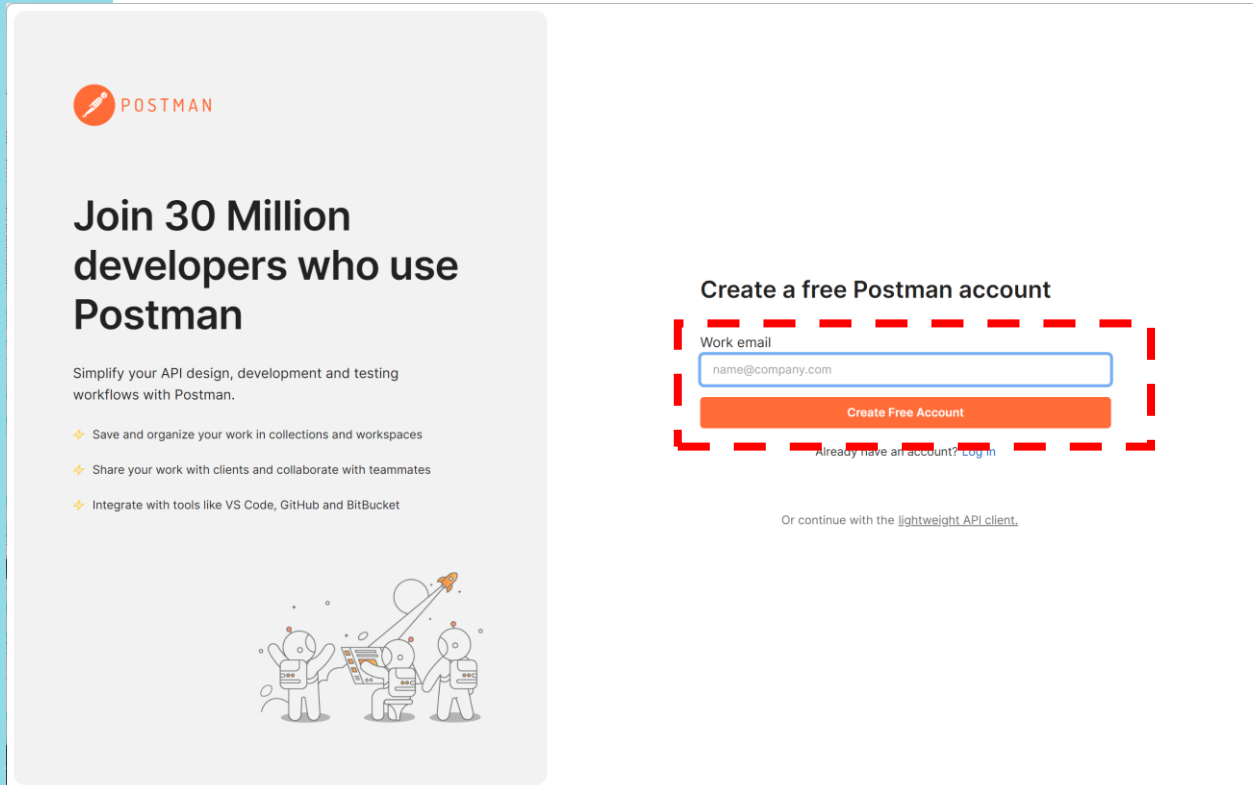
- Postman เป็นเครื่องมือที่เอาไว้ทดสอบการทำงานของ API โดยที่ไม่จำเป็นต้องมี Front-End หรือเขียนเว็บในฝั่งหน้าบ้าน
- สามารถเข้าไปติดตั้งได้ที่ <https://www.postman.com/downloads/>



ติดตั้ง Postman (1)

63

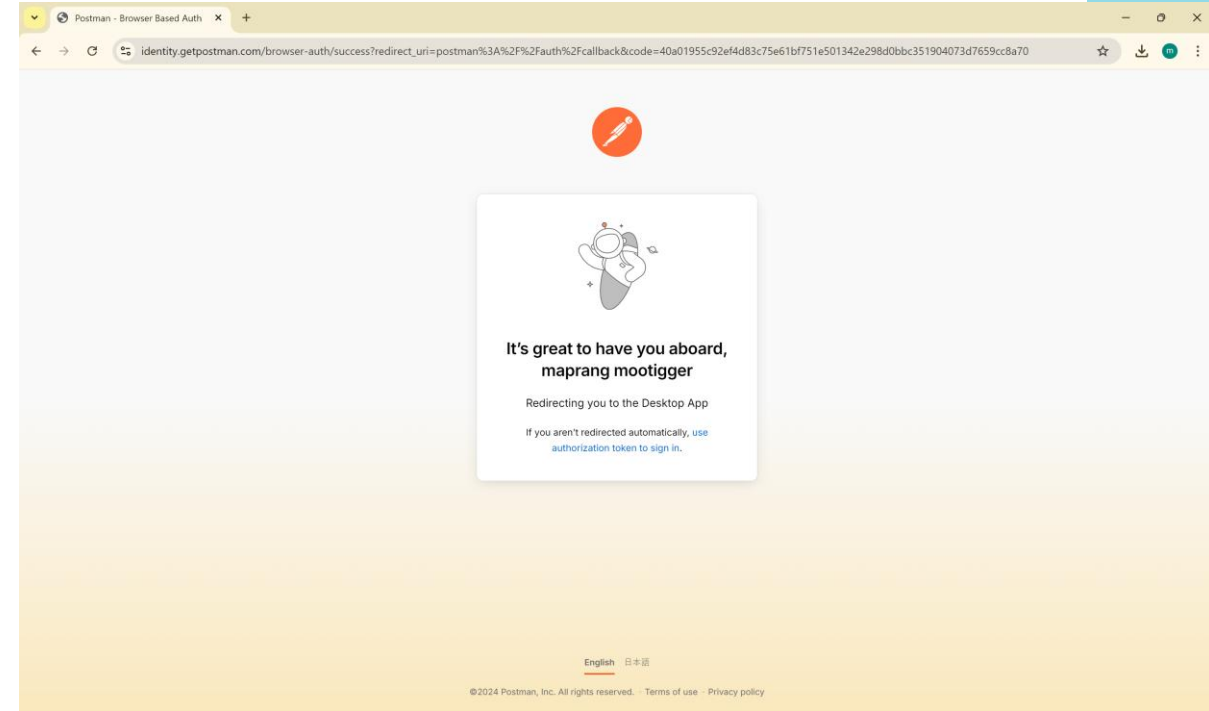
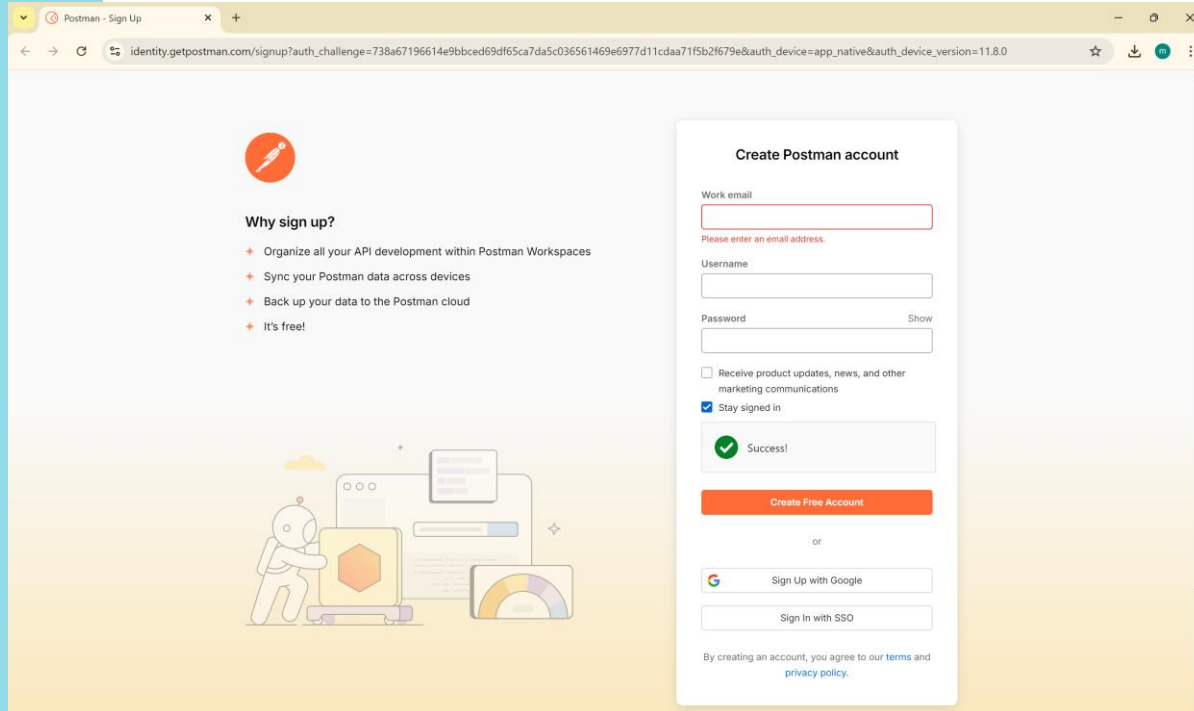
- ที่ <https://www.postman.com/downloads/>



ติดตั้ง Postman (2)

64

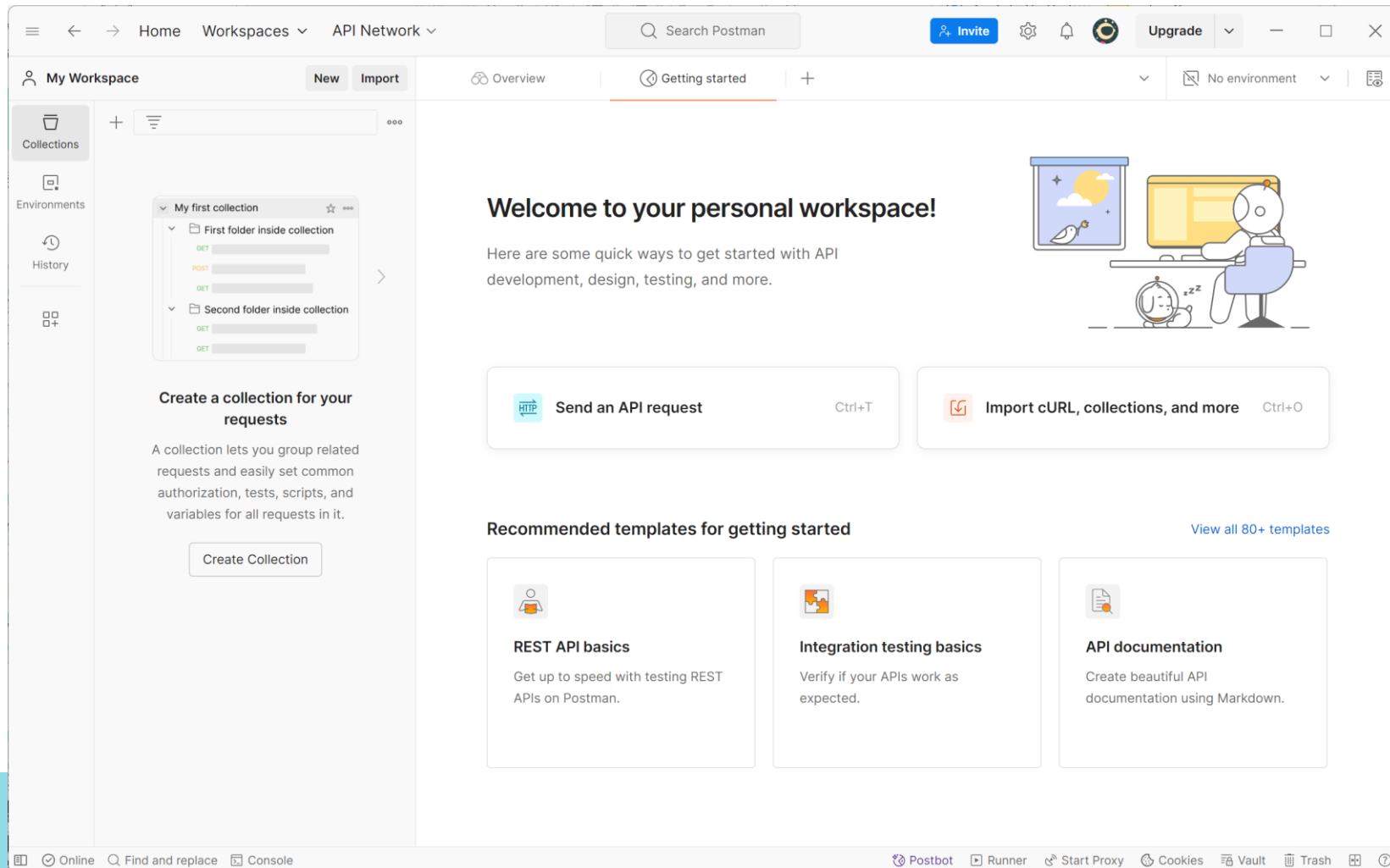
- โปรแกรมจะลิงค์ไปหน้าเว็บเพื่อให้สร้างบัญชี Postman



ติดตั้ง Postman (3)

65

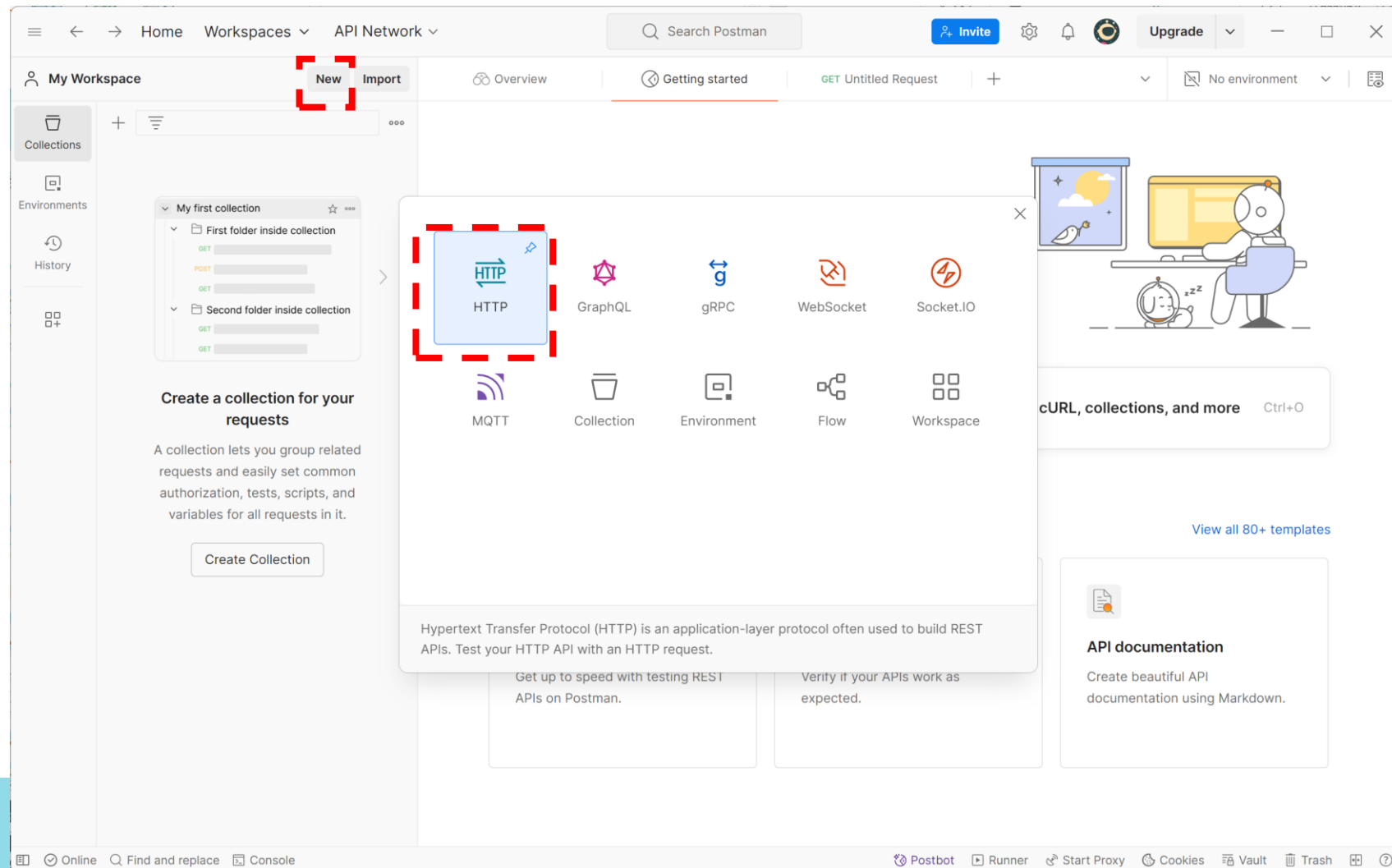
- เมื่อสร้างบัญชี Postman แล้วลงชื่อเข้าใช้ โปรแกรมจะแสดงหน้าเริ่มต้น



ติดตั้ง Postman (4)

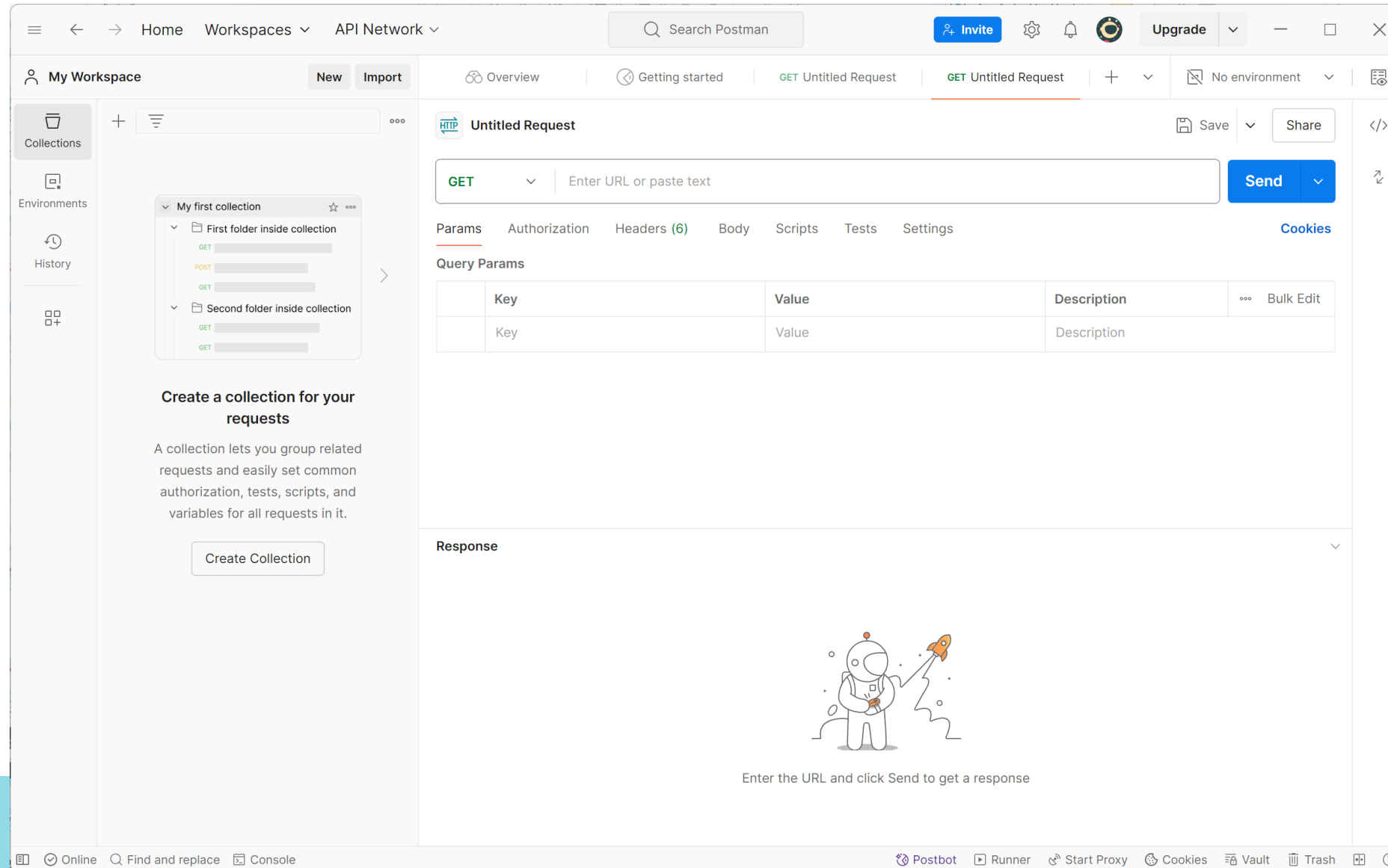
66

- โปรแกรมเปิดหน้าเริ่มต้น



เริ่มต้นใช้งาน Postman (1)

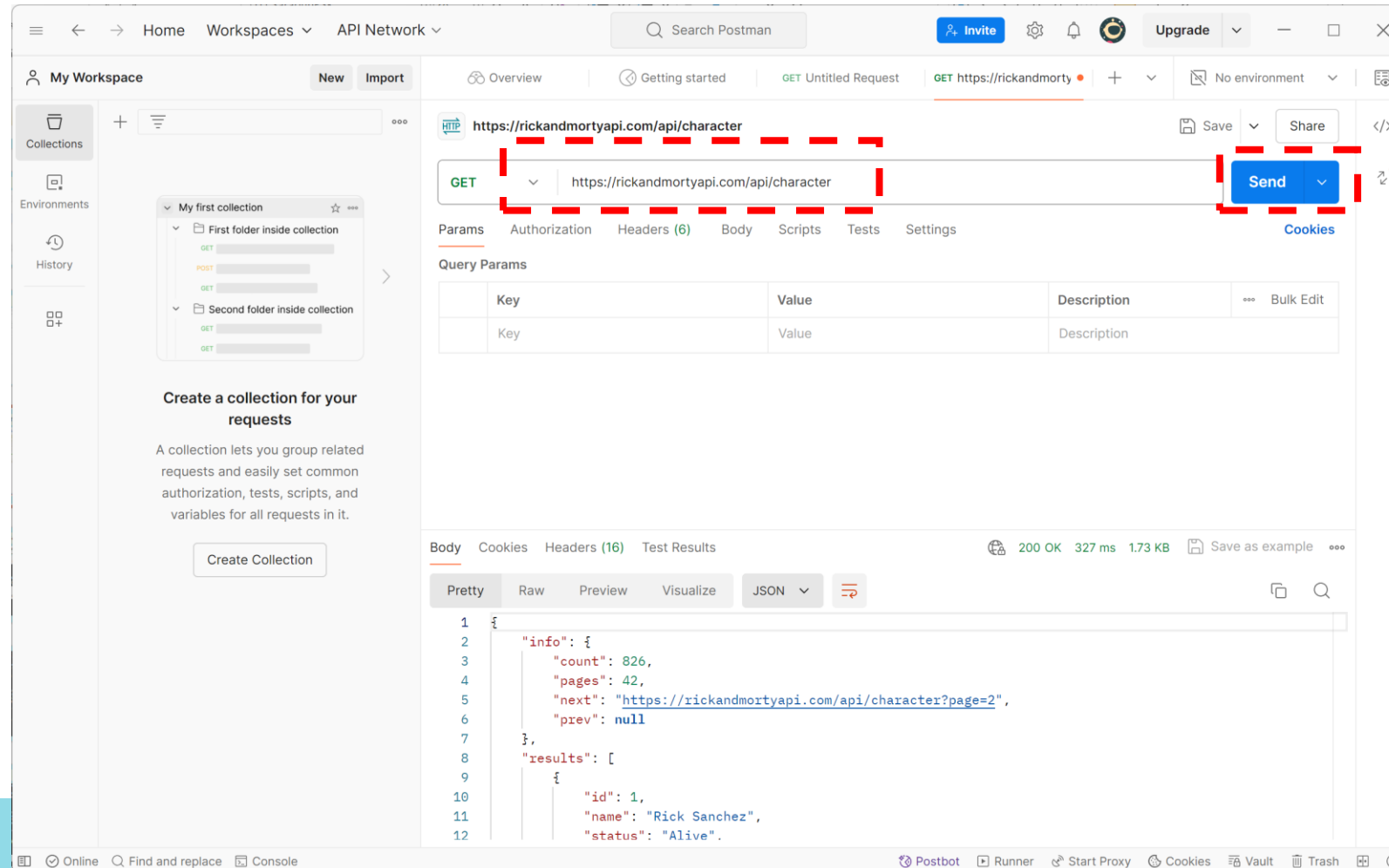
67



เริ่มต้นใช้งาน Postman (2)

68

- ตัวอย่าง API ที่นำมาใช้งานในตัวอย่าง: <https://rickandmortyapi.com/api/character>



ขอข้อมูลแบบ GET ด้วย Fetch API

69

- fetch() เป็น method ที่ให้ทำให้สามารถ รับ-ส่ง ข้อมูล (HTTP Request) ระหว่างเว็บได้จากเว็บเบราว์เซอร์
ตัว Fetch API จะ return ค่า Promise กลับมา
- รูปแบบ syntax ของ Fetch API วิธีการ error handling จะใช้ .then() และ .catch (err) {} ดังนี้

```
fetch(url)
  .then(res => res.json())
  .then(json => console.log(json))
  .catch(error => console.log(error)) // handle the error
```

- หรือแบบ async/await วิธีการ error handling จะใช้ try {}catch (err) {}

```
//Get method
const res = await fetch(url)
const json = await res.json()
console.log(json)
```

ค่าที่ fetch() ส่งกลับมา เป็น object จะมีค่าข้างในหลักๆ คือ

- ok - เป็นค่า true หรือ false
- status - ค่า HTTP status code ที่ได้ (ปกติ จะเป็น 200)
- statusText - ค่า default

- fetch() เป็น method สำหรับส่ง HTTP request (GET/POST/PUT/DELETE ฯลฯ) คืนค่าเป็น Promise<Response> ใช้ได้ในเบราว์เซอร์ และใน Node.js เวอร์ชันใหม่ (Node $\geq$ 18)
- โครงสร้างพื้นฐาน

```
const res = await fetch(url, {  
  method: 'GET',           // ค่าเริ่มต้นคือ GET (GET/POST/PUT/DELETE ฯลฯ)  
  headers: { /* ... */ },  // เช่น Content-Type, Authorization  
  body: JSON.stringify(obj), // แปลงค่า (obj) ให้เป็น สตริง JSON ใช้กับ POST/PUT/PATCH  
});
```

ขอข้อมูลแบบ GET ด้วย Fetch API

71

- ขอข้อมูลแบบ GET ด้วย Fetch API แล้วแปลงผลลัพธ์เป็น JSON

```
async function fetchData() {  
  try {  
    const url = `https://.....`  
    const res = await fetch(url);  
    if (!res.ok) {  
      throw new Error('Network response was not ok');  
    }  
    const json = await res.json();  
    console.log(json);  
  } catch (error) {  
    console.error('Fetch error:', error);  
  }  
}
```

- json() คือ เมธอดที่ทำหน้าที่ อ่าน body แล้ว แปลง (parse) เป็น JSON คืนค่าเป็น Promise

ตัวอย่างการใช้ Fetch API เพื่อดึงข้อมูลมาใช้งาน (1)

- API ที่นำมาใช้งานในตัวอย่าง:
<https://rickandmortyapi.com/>
- ก่อนใช้งาน API ควรทำความเข้าใจ API นั้น ๆ ก่อนใช้งาน
- Rick and Morty API คือ API ที่อ้างอิงจากรายการโทรทัศน์ Rick and Morty สามารถเข้าถึงข้อมูลของตัวละคร รูปภาพ สถานที่ และตอนต่างๆ
- ถ้าต้องการค้นหาข้อมูลตัวละครด้วยชื่อ แล้วดึงข้อมูล name, image, gender มาแสดงที่หน้าเว็บมีตัวอย่างดังนี้

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Rick and Morty API</title>
</head>
<body>
  <h1>ตัวละครในการ์ตูน Rick and Morty </h1>
  <input class="user_input" id="TextInput" type="text"
name="mytext" placeholder="ใส่ชื่อตัวละคร">
  <input type="button" value="ค้นหา" id="go-button">

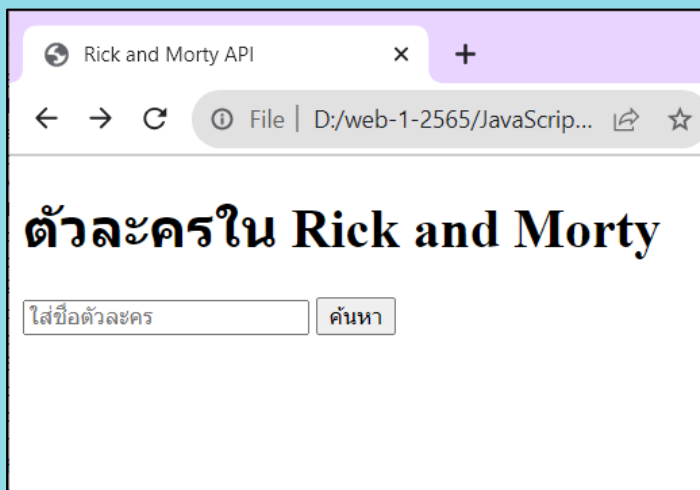
  <div class="card">

  </div>
  <script src="RickAndMorty.js" charset="utf-8"></script>

</body>
</html>
```

ตัวอย่างการใช้ Fetch API เพื่อดึงข้อมูลมาใช้งาน (2)

73



```
<!DOCTYPE html>
<head>
  <meta charset="UTF-8">
  <title>Rick and Morty API</title>

</head>
<body>
  <h1>ตัวละครใน Rick and Morty </h1>
  <input class="user_input" id="TextInput" type="text"
name="mytext" placeholder="ใส่ชื่อตัวละคร">
  <input type="button" value="ค้นหา" id="go-button">

  <div id="card">

  </div>
  <script src="RickAndMorty.js" charset="utf-8"></script>

</body>
</html>
```

rick&morty.html

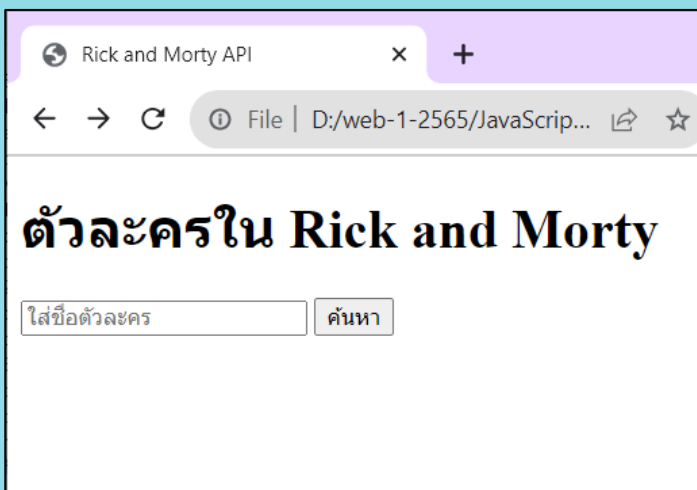
ตัวอย่างการใช้ Fetch API เพื่อดึงข้อมูลมาใช้งาน (3)

74

RickAndMorty.js

```
const userInput = document.getElementById('TextInput')//เลือกกรอกข้อมูล
จาก id
const btn = document.getElementById("go-button");//เลือกปุ่มกดจาก id
const divCard = document.getElementsByClassName("card");//เลือก<div>
จาก class

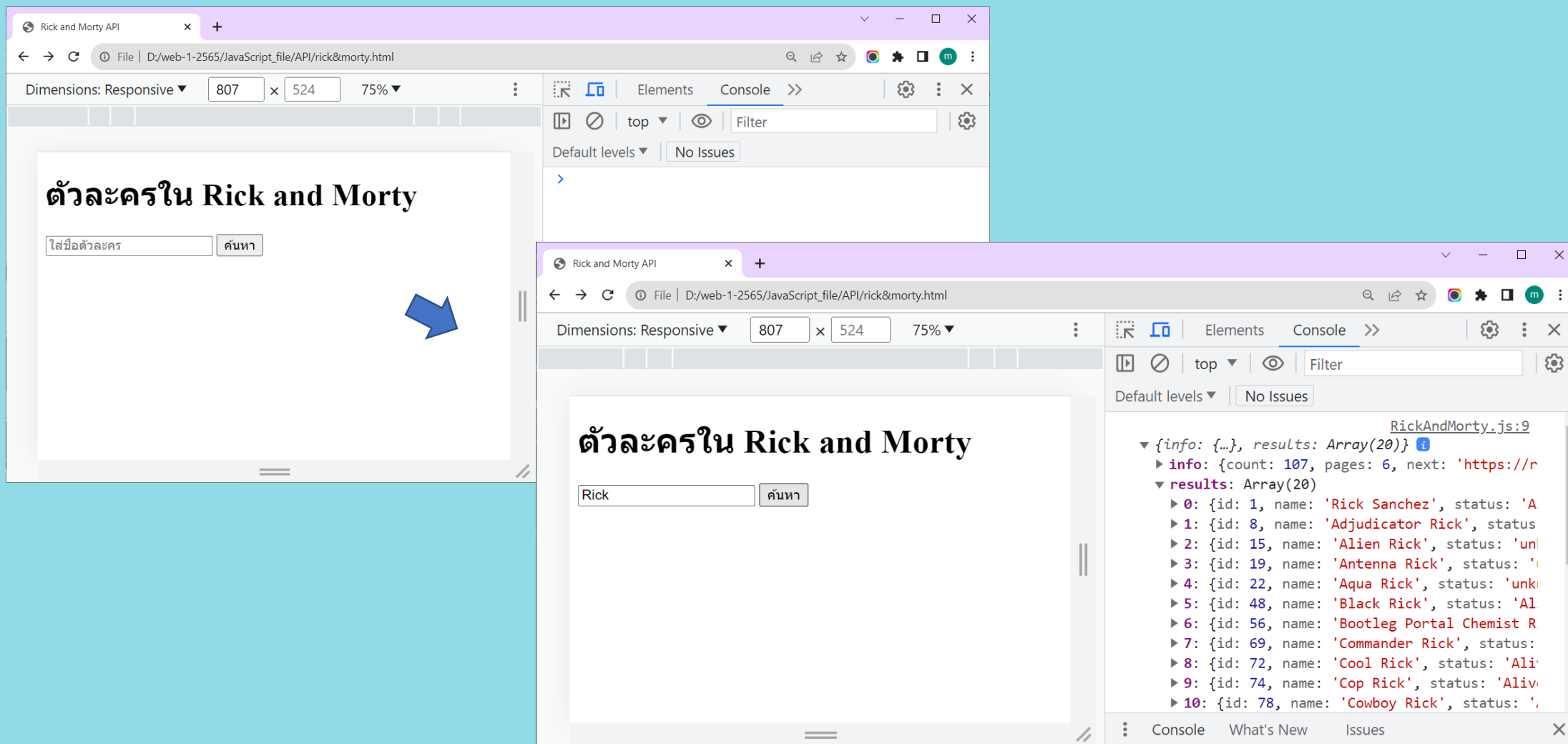
btn.addEventListener("click", async function buttonClicked() { //
กำหนดเหตุการณ์เมื่อกดปุ่ม
    const getName = userInput.value//ดึงค่าจากช่องกรอกข้อมูล
    const url =
`https://rickandmortyapi.com/api/character/?name=${getName}`
    const res = await fetch(url)
    const json = await res.json()
    console.log(json) //ดูผลลัพธ์การดึง api
})
```



ที่ url เมื่อต้องการดึง api โดยการระบุชื่อ คู่มือการใช้งานให้กำหนดการใช้ url ตามด้วย ? และตามด้วย <query>=<value> (ต้องการดึงข้อมูลด้วยชื่อนั้นแทน <query> ด้วยพารามิเตอร์ name ส่วนค่า <value> คือชื่อตัวละคร)
ดูเพิ่มเติมที่ <https://rickandmortyapi.com/documentation/#filter-characters>

ตัวอย่างการใช้ Fetch API เพื่อดึงข้อมูลมาใช้งาน (4)

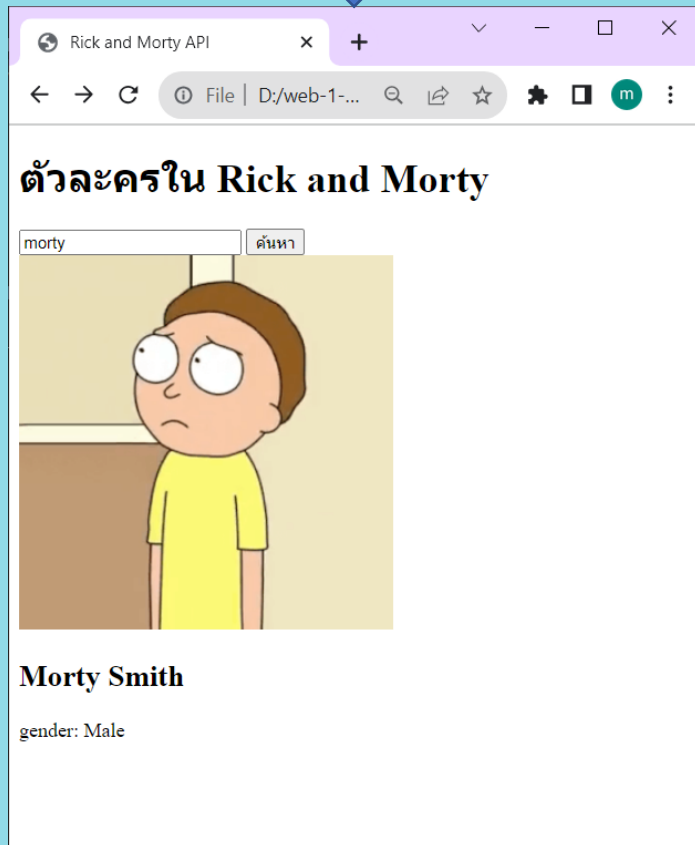
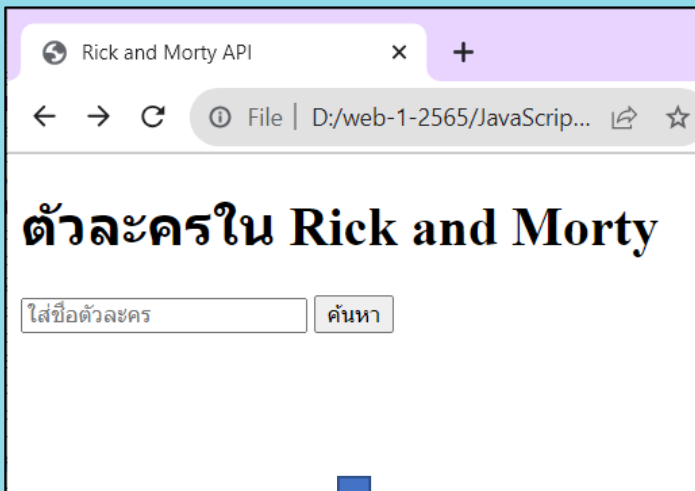
75



เมื่อทดลองกรอก Rick แล้วดูผลลัพธ์การดึง api ที่ console จะเห็นว่ามีผลลัพธ์ (results) หลายตัว ดังนั้นเวลาใช้งานควรระบุว่าจะใช้
งานผลลัพธ์ที่ตัวใด

ตัวอย่างการใช้ Fetch API เพื่อดึงข้อมูลมาใช้งาน (5) ⁷⁶

RickAndMorty.js



```
const userInput = document.getElementById('TextInput') // เลือก id ของ input
const btn = document.getElementById("go-button"); // เลือกปุ่มกดจาก id
const divCard = document.getElementById("card")
```

```
btn.addEventListener("click", async function buttonClicked() { // กำหนดเหตุการณ์เมื่อคลิกปุ่ม
  const getName = userInput.value // ดึงค่าจากช่องกรอกข้อมูล
  const url = `https://rickandmortyapi.com/api/character/?name=${getName}`
  const res = await fetch(url)
  const json = await res.json()
  console.log(json) // ดูผลลัพธ์การดึง api
```

```
const character = json.results[0] // เลือกข้อมูล api ตัวแรก
```

```
const characterImg = document.createElement('img') // สร้าง <img>
characterImg.src = character.image // กำหนด src ให้ <img>
divCard.appendChild(characterImg) // เอา <img> ใสใน div ที่ id=card
```

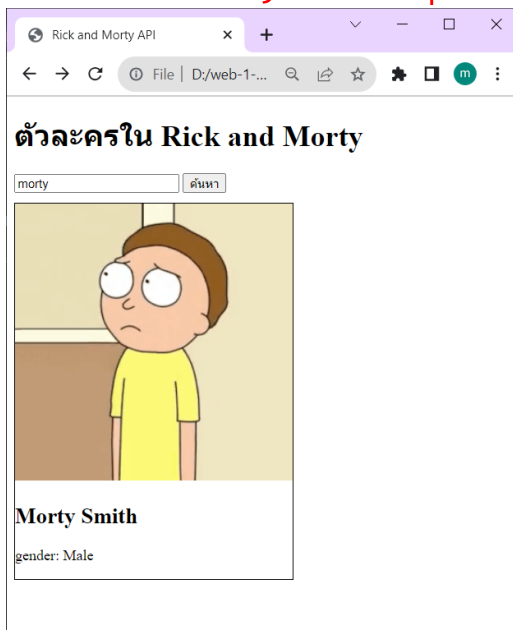
```
const characterName = document.createElement('h2'); // สร้าง <h2>
characterName.innerText = character.name // กำหนดข้อความภายใน <h2>
divCard.appendChild(characterName) // เอา <h2> ใสใน div ที่ id=card
```

```
const characterGender = document.createElement('p'); // สร้าง <p>
characterGender.innerText = `gender: ${character.gender}` // กำหนดข้อความภายใน <p>
divCard.appendChild(characterGender) // เอา <p> ใสใน div ที่ id=card
})
```

กรอกข้อมูล แล้วกดปุ่ม
จะดึงข้อมูลมาแสดงโดย
ใช้ results[0] (ข้อมูลตัว
แรก) และดึง รูป ชื่อ
เพศ มาใช้และสร้างแท็ก
ต่างในเพื่อแสดงข้อมูลที่
หน้าเว็บ

ตัวอย่างการใช้ Fetch API เพื่อดึง ข้อมูลมาใช้งาน (6)

- ตกแต่งให้สวยงามโดยใช้ CSS โดยกำหนด style ใน html แล้วเรียกใช้ใน javascript



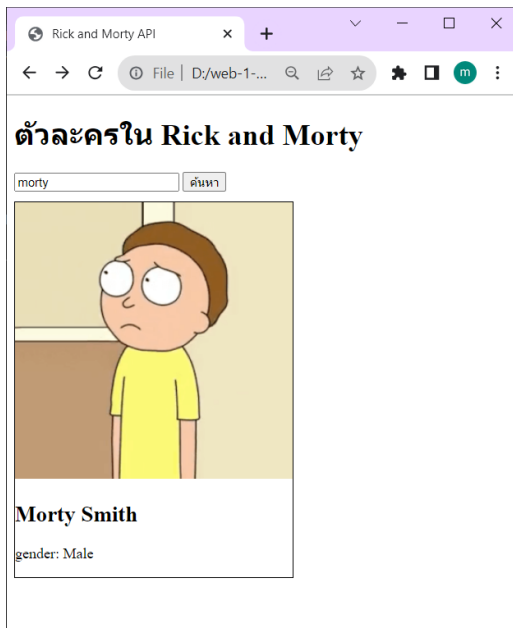
rick&morty.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Rick and Morty API</title>
  <style>
    .style1{
      margin-top: 10px;
      border: 1px solid black;
      display: inline-block; //อยากให้ขนาด div พอดีข้อความ
      text-align: center;
    }
  </style>
</head>
<body>
  <h1>ตัวละครใน Rick and Morty </h1>
  <input class="user_input" id="TextInput" type="text" name="mytext"
placeholder="ใส่ชื่อตัวละคร">
  <input type="button" value="ค้นหา" id="go-button">
  <br>
  <div id="card">

  </div>
  <script src="RickAndMorty.js" charset="utf-8"></script>
</body>
</html>
```

ตัวอย่างการใช้ Fetch API เพื่อดึง ข้อมูลมาใช้งาน (7)

- ตกแต่งให้สวยงามโดยใช้ CSS โดยกำหนด style ใน html แล้วเรียกใช้ใน javascript



```
const userInput = document.getElementById('TextInput')//เลือกกรอกข้อมูลจาก id
const btn = document.getElementById("go-button");//เลือกปุ่มกดจาก id
const divCard = document.getElementById("card")

btn.addEventListener("click", async function buttonClicked() { //กำหนดเหตุการณ์เมื่อกดปุ่ม
  const getName = userInput.value//ดึงค่าจากช่องกรอกข้อมูล
  const url = `https://rickandmortyapi.com/api/character/?name=${getName}`
  const res = await fetch(url)
  const json = await res.json()
  console.log(json) //ดูผลลัพธ์การดึง api

  const character = json.results[0] //เลือกข้อมูล api ตัวแรก

  divCard.className = 'style1' //ตกแต่งด้วย CSS

  const characterImg = document.createElement('img')//สร้าง <img>
  characterImg.src = character.image//กำหนด src ให้<img>
  divCard.appendChild(characterImg)//เอา <img> ใส่ใน div ที่ id=card

  const characterName = document.createElement('h2'); //สร้าง <h2>
  characterName.innerText = character.name//กำหนดข้อความภายใน <h2>
  divCard.appendChild(characterName)//เอา <h2> ใส่ใน div ที่ id=card

  const characterGender= document.createElement('p'); //สร้าง <p>
  characterGender.innerText = `gender: ${character.gender}`//กำหนดข้อความภายใน <p>
  divCard.appendChild(characterGender)//เอา <p> ใส่ใน div ที่ id=card
})
```

ตัวอย่างการใช้ Fetch API เพื่อดึง ข้อมูลมาใช้งาน (8)

- ใช้ try {} catch (err) {}
ในการจัดการกับ
ข้อผิดพลาดที่อาจจะ
เกิดขึ้น

```
const userInput = document.getElementById('TextInput');//เลือกกรอกข้อมูลจาก id
const btn = document.getElementById("go-button");//เลือกปุ่มกดจาก id
const divCard = document.getElementById("card")

btn.addEventListener("click", async function buttonClicked() { //กำหนดเหตุการณ์เมื่อกดปุ่ม
  try{
    const getName = userInput.value//ดึงค่าจากช่องกรอกข้อมูล
    const url = `https://rickandmortyapi.com/api/character/?name=${getName}`
    const res = await fetch(url)
    const json = await res.json()
    console.log(json) //ดูผลลัพธ์การดึง api
    if(!json.results) {//เช็คค่า json.results ไม่มีค่า
      throw {message:`${getName}ไม่มีตัวละครนี้`};
    }

    const character = json.results[0] //เลือกข้อมูล api ตัวแรก

    divCard.className = 'style1' //ตกแต่งด้วย CSS

    const characterImg = document.createElement('img');//สร้าง <img>
    characterImg.src = character.image//กำหนด src ให้ <img>
    divCard.appendChild(characterImg)//เอา <img> ใส่ใน div ที่ id=card

    const characterName = document.createElement('h2');//สร้าง <h2>
    characterName.innerText = character.name//กำหนดข้อความภายใน <h2>
    divCard.appendChild(characterName)//เอา <h2> ใส่ใน div ที่ id=card

    const characterGender= document.createElement('p');//สร้าง <p>
    characterGender.innerText = `gender: ${character.gender}`//กำหนดข้อความภายใน <p>
    divCard.appendChild(characterGender)//เอา <p> ใส่ใน div ที่ id=card

  }catch(error){
    window.alert(error.message)//แสดงข้อความจาก throw
  }finally{
    userInput.value=""//ลบค่าในช่องกรอกข้อมูล
  }
})
```

RickAndMorty.js 7

ตัวอย่างการใช้ Fetch API เพื่อดึง ข้อมูลมาใช้งาน (9)

- แสดงผลหลังจากใช้ try
{catch (err) {} ในการจัดการ
กับข้อผิดพลาดที่อาจจะเกิดขึ้น

RickAndMorty.js 8

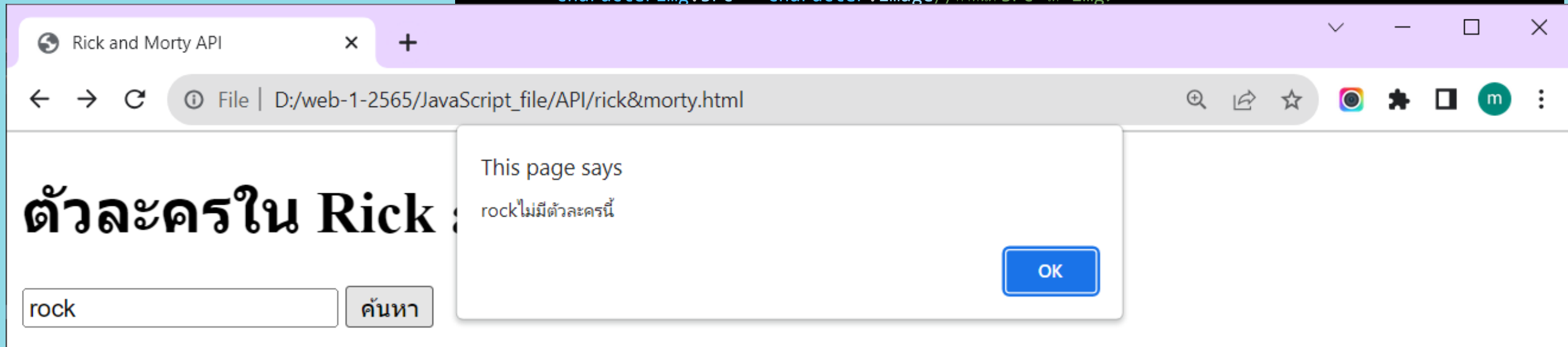
```
const userInput = document.getElementById('TextInput')//เลือกกรอกข้อมูลจาก id
const btn = document.getElementById("go-button");//เลือกปุ่มกดจาก id
const divCard = document.getElementById("card")

btn.addEventListener("click", async function buttonClicked() { //กำหนดเหตุการณ์เมื่อกดปุ่ม
  try{
    const getName = userInput.value//ดึงค่าจากช่องกรอกข้อมูล
    const url = `https://rickandmortyapi.com/api/character/?name=${getName}`
    const res = await fetch(url)
    const json = await res.json()
    console.log(json) //ดูผลลัพธ์การดึง api
    if(!json.results) {//เช็คค่า json.results ไม่มีค่า
      throw {message:`${getName}ไม่มีตัวละครนี้`};
    }

    const character = json.results[0] //เลือกข้อมูล api ตัวแรก

    divCard.className = 'style1' //ตกแต่งด้วย CSS

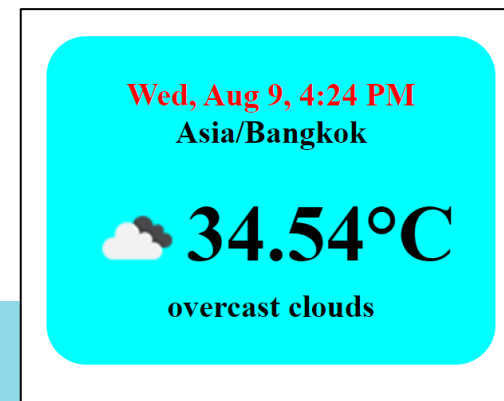
    const characterImg = document.createElement('img')//สร้าง <img>
    characterImg.src = character.image//กำหนด src ให้ <img>
```



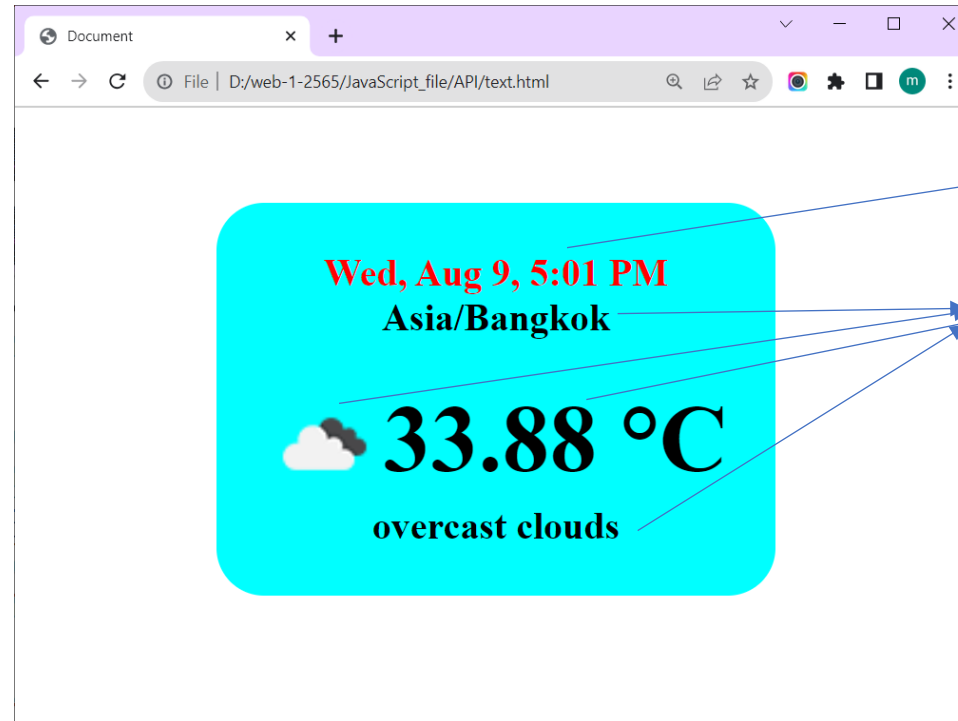
- JavaScript-Get API
 - Laboratory 7



- ดึงข้อมูลสภาพอากาศของกรุงเทพ ที่เว็บ <https://openweathermap.org/api/one-call-3#start>
- ให้นักศึกษาเข้าเว็บไซต์ดังกล่าวแล้วดูวิธีการใช้ api เพื่อเอาข้อมูลสภาพอากาศปัจจุบัน (Current weather data)
 - ดึงชื่อ icon แล้วดึงรูปผ่านลิงค์ <https://openweathermap.org/img/wn/ชื่อicon.png>
 - ดึงข้อมูล description
 - ดึงข้อมูลอุณหภูมิ (เซลเซียส °C) วิธีการพิมพ์ ° ให้กด Alt(ค้าง) + 0176
 - ดึง timezone
- โดยกรุงเทพมี Latitude=13.736717 และ Longitude= 100.523186 (ข้อมูลจาก google)
- API key คือ f5aff27b8a2f72928b971c9e2b08b4cc
- สร้างหน้าเว็บแสดงผลการดึงข้อมูลดังรูป



- สร้างหน้าเว็บแสดงผลการดึงข้อมูลดังรูป ข้อมูลในรูปจะมีการเปลี่ยนแปลงข้อมูลตาม API



วันเวลาที่ได้จาก date()

ดึงข้อมูล API มาใช้

ข้อมูลบรรทัดที่ 1 แสดงวัน เดือน วันที่ และเวลา เป็นการดึงข้อมูล ณ ขณะนั้นโดยใช้ date() ดูการใช้สไลด์หน้า 11

ข้อมูลบรรทัดที่ 2 timezone ของกรุงเทพเป็นข้อมูล api จากเว็บ openweathermap.org

ข้อมูลบรรทัดที่ 3 รูป และตัวเลขอุณหภูมิ เป็นข้อมูล api จากเว็บ openweathermap.org

ข้อมูลบรรทัดที่ 4 description เป็นข้อมูล api จากเว็บ openweathermap.org